Finding an Edge in Prediction Markets

ML-based Jump Detection on Kalshi

Group #51

Lennard Pische
John A. Paulson School
of Engineering and Applied Sciences
Harvard University
Cambridge, MA 02138
lenny_pische@
college.harvard.edu

Gianluca Pisa
MIT Sloan School of Management
Massachusetts Institute of Technology
Cambridge, MA 02142
gpisa@mit.edu

Andres Blanco Prada
MIT Sloan School of Management
Massachusetts Institute of Technology
Cambridge, MA 02142
abprada@mit.edu

Moritz Wassermann
MIT Sloan School of Management
Massachusetts Institute of Technology
Cambridge, MA 02142
mwassermann@fas.harvard.edu

Vishwesh Venkatramani
MIT Sloan School of Management
Massachusetts Institute of Technology
Cambridge, MA 02142
vv29@mit.edu

Abstract

We investigate whether trade microstructure, market identity, and recent price history can predict the direction of large price jumps in Kalshi prediction markets. Using 62 million trades from 2025, we frame the problem as three-class classification, upward jump, downward jump, or no jump, across four forecast horizons (5, 15, 30, and 60 minutes). We train and evaluate six model families: LightGBM, LSTM, Mamba, a pretrained Moirai backbone, a CNN-Tokenized Transformer (CTTS), and a Feature-Tokenizer Transformer (FT-Transformer), combined via a learned soft Mixture-of-Experts (MoE) gating network. All models achieve out-of-sample macro F1 in the 0.47–0.54 range, meaningfully above chance but well below thresholds required for systematic profitable trading. LightGBM, operating on 36 engineered causal features, matches or leads every sequence architecture at every horizon, consistent with recent tabular deep-learning literature showing that well-specified rolling-window features are difficult for sequence models to improve upon. The MoE gate routes approximately 50% of weight to LightGBM and fails to escape the directional-confusion ceiling shared by all six experts. We release a detailed analysis of feature importance, gate routing dynamics, and per-class error structure, and identify limit order book integration and two-stage jump detection as the most promising directions for future work.

Working paper. Advanced Topics in Data Science, Harvard University, 2026.

1 Background and Motivation

Prediction markets trade binary contracts whose prices represent market-implied probabilities of real-world events. On Kalshi, a YES contract priced at 70 cents corresponds to a 70% implied probability; each contract resolves to either \$1 or \$0, so price changes directly reflect shifts in collective beliefs about outcomes.

Prices move sharply around news events, sports outcomes, and macroeconomic releases. For traders and liquidity providers, anticipating such moves supports position sizing, risk management, and market-making decisions. As Kalshi's liquidity deepens, with 86.1% of all 2021–2025 trades concentrated in 2025 alone, so does the value of extracting predictive signal from trade microstructure.

Every Kalshi contract is inherently directional: a trader must predict not only that a large move is coming, but whether the YES price will jump up or down. This motivates our three-class formulation distinguishing upward jumps, downward jumps, and no-jump outcomes across four forecast horizons.

From an ML perspective, this setting poses a non-trivial modeling challenge. Labels are heavily imbalanced, jump events constitute only 8–11% of trades per direction, and the signal-to-noise ratio is low in an increasingly competitive market. A central empirical question is whether complex sequence architectures offer meaningful gains over carefully engineered tabular baselines. Our results suggest they largely do not, a finding with direct implications for practitioners deciding where to allocate modeling effort in similar high-frequency event-driven settings.

2 Problem Statement

Our central research question is:

> *Can trade history, market identity, and recent market microstructure be used to predict the direction of large future price jumps in Kalshi prediction markets?*

The unit of observation is an executed trade t on a given Kalshi ticker. At each trade, the model observes only information available up to that timestamp, including the current price, volume, recent trading activity, historical returns, and a semantic representation of the contract ticker. For each forecast horizon $h \in \{5, 15, 30, 60\}$ minutes, the target is a three-class label:

$$y_t^{(h)} \in \{-1, 0, +1\},$$

where -1 denotes a downward jump, 0 denotes no jump, and $+1$ denotes an upward jump in the YES price over the next h minutes.

The model output is a horizon-specific probability vector,

$$P\left(y_t^{(h)} = -1\right), \quad P\left(y_t^{(h)} = 0\right), \quad P\left(y_t^{(h)} = +1\right),$$

which can be interpreted as the predicted likelihood of a downward jump, no jump, or upward jump. The empirical task is to determine whether these probabilities contain reliable signal out of sample, and whether more complex sequence models improve on strong tabular baselines built from causal features.

3 Data & EDA

3.1 Dataset

The dataset contains 72 million trade-level records from the Kalshi prediction market exchange, spanning 30 June 2021 through 25 November 2025 (Becker 2026). Each record captures six fields: a structured `ticker` encoding market series, resolution date, and outcome priced; `yes_price` and `no_price` in cents on $[0, 100]$, representing complementary implied probabilities; `count`, the number of contracts exchanged; `taker_side`, indicating which side initiated the trade; and `created_time`, a UTC microsecond timestamp.

3.2 Sample Restriction

Trading activity is severely right-skewed over time: despite covering over four years, 86.1% of all trades occur after 2025-01-01. Prior to this, volume is too sparse and episodic to support reliable sequence modeling or stable feature estimation. We therefore restrict the analysis to the 2025 calendar year, retaining 62 million trades while discarding only 13.9% of records but 79.5% of calendar coverage.

3.3 Exploratory Observations

Three properties of the data directly inform the modeling approach. First, trades cluster around news events and resolution deadlines, producing highly irregular inter-trade spacing. This motivates log-transforming time deltas and using activity-rate features rather than fixed-frequency aggregations. Second, the `yes_price` distribution is multimodal and contract-dependent, reflecting the heterogeneous nature of Kalshi markets across asset classes (crypto, politics, sports, macro). This motivates the semantic ticker embeddings described in Section 4.4, which allow the model to share statistical strength across thematically related contracts. Third, the forward return distribution is sharply zero-peaked and heavy-tailed at all horizons, ruling out standard regression targets and motivating the conditional quantile thresholding used to define jump labels in Section 4.1.2.

4 Methods & Models

4.1 Target Construction

We define two complementary targets at each horizon $h \in \{5, 15, 30, 60\}$ minutes: a continuous pseudo-target measuring price-move magnitude and a discrete three-class jump label.

4.1.1 Forward Returns

For trade t on ticker T with timestamp τ_t and yes-price p_t, let $t^\star$ be the most recent trade on ticker T with $\tau_{t^\star} \leq \tau_t + h$ minutes. The signed forward return is

$$\Delta p_t^{(h)} = p_{t^\star} - p_t.$$

If no successor trade exists in $(\tau_t,\, \tau_t + h]$, we set $t^\star = t$ and $\Delta p_t^{(h)} = 0$. We use additive price differences rather than percentage returns: on the bounded $[0, 100]$ scale, percentage returns are ill-defined near zero and compressed near 100.

4.1.2 Train/Test Split and Threshold Selection

We adopt a strict temporal split: the first 80% of trades form the training set and the remaining 20% the test set. All thresholding decisions use only the training distribution. Directional thresholds are defined as conditional quantiles of the training signed returns:

$$\text{up\_thr}^{(h)} = Q_{1-\alpha}\big(\Delta p_t^{(h)} \;\big|\; \Delta p_t^{(h)} > 0,\, t \in \text{train}\big), \quad \text{down\_thr}^{(h)} = Q_{\alpha}\big(\Delta p_t^{(h)} \;\big|\; \Delta p_t^{(h)} < 0,\, t \in \text{train}\big),$$

with $\alpha = 0.25$. Conditioning on sign before taking the quantile is necessary because the spike at $\Delta p = 0$ is large; an unconditional quantile would collapse to zero and yield degenerate labels.

Figure 1 shows the empirical signed-return distributions across horizons. Distributions are sharply zero-peaked and heavy-tailed, with tails broadening monotonically from roughly ± 10 cents at 5 minutes to $-28/+31$ cents at 60 minutes.

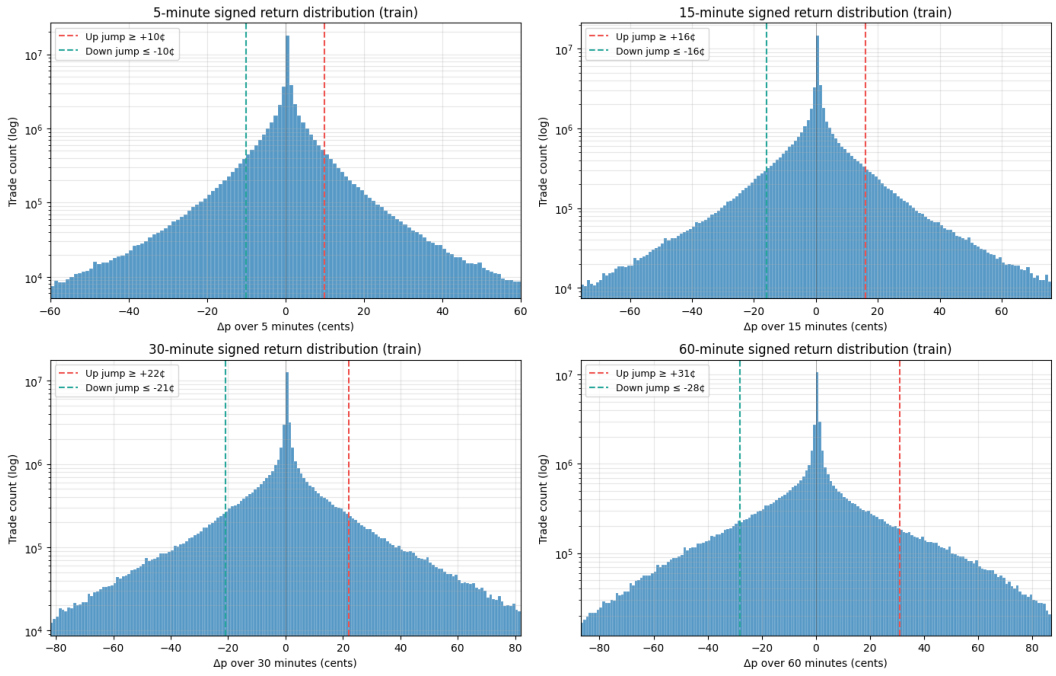

Figure 1: Distribution of forward price changes across prediction horizons. Dashed lines mark the directional thresholds used to define jump labels.

4.1.3 Three-class Jump Labels

Given the thresholds, each trade is assigned:

$$y_t^{(h)} = \begin{cases} +1 & \text{if } \Delta p_t^{(h)} \geq \text{up\_thr}^{(h)}, \\ -1 & \text{if } \Delta p_t^{(h)} \leq \text{down\_thr}^{(h)}, \\ 0 & \text{otherwise.} \end{cases}$$

The same train-derived thresholds are applied to both splits, so test-set class definitions are independent of test-set statistics. Table 1 reports the resulting class distributions. Jump rates are roughly 8–10% per direction in training, with test-set rates consistently higher by approximately one percentage point per direction, indicating modestly elevated volatility in the held-out period. The up/down balance is preserved across splits, consistent with a mild symmetric uplift in volatility rather than a structural distributional shift.

Table 1: Class distributions for three-class jump labels $y_t^{(h)}$ across horizons and splits.

Horizon	**down_thr**	**up_thr**	**Train (%)**		**Test (%)**		
(min)	(¢)	(¢)	down	up	down	up	no-jump (train / test)
5	−10	+10	8.33	8.39	9.17	9.15	83.27 / 81.68
15	−16	+16	9.29	9.34	10.16	10.04	81.37 / 79.80
30	−21	+22	10.07	9.63	11.16	10.55	80.31 / 78.29
60	−28	+31	10.59	9.64	11.83	10.60	79.77 / 77.57

The model is trained jointly on both target families: a regression head supervises the continuous $\Delta p_t^{(h)}$, while a classification head targets the categorical $y_t^{(h)}$, allowing a single forward pass to produce horizon-specific predictions.

4.2 Models

4.2.1 Light Gradient Boosting Machine

LightGBM operates directly on the 36-dimensional engineered feature vector, using leaf-wise tree growth to capture non-linear interactions between backward returns, jump counts, price level, and time-of-day without temporal windowing. Four independent models are trained per horizon. Full details are in Appendix B.

4.2.2 Long Short-Term Memory

The LSTM processes windows of 32 consecutive same-ticker trades, using gated hidden states to selectively retain information across timesteps. It serves as the canonical sequential baseline for event-driven market data. Full details are in Appendix C.

4.2.3 Mamba

Mamba Gu and Dao 2023 is a selective state-space classifier trained from scratch on 32-trade windows. Its input-dependent gating compresses uninformative trades cheaply while updating aggressively on genuine price shocks, an inductive bias well matched to Kalshi's bursty trade sequences. Full details are in Appendix D.

4.2.4 Moirai

Moirai Woo et al. 2024 is the only pretrained model in this study. The native forecasting head is replaced with a multi-horizon classification head; the backbone is fine-tuned at a $30\times$ lower learning rate than the head to preserve pretrained representations. It serves as a probe of whether general time-series pretraining transfers to prediction-market microstructure. Full details are in Appendix E.

4.2.5 CNN-Tokenized Transformer Sequence Classifier

CTTS Zeng et al. 2023 uses a 1D CNN to compress 64-trade windows into 16 dense tokens, then applies full self-attention over those tokens via a Transformer encoder. The CNN captures local microstructure bursts; the Transformer captures long-range cross-burst dependencies. Full details are in Appendix F.

4.2.6 Feature-Tokenizer Transformer

The FT-Transformer Gorishniy et al. 2021 tokenizes each of the 36 engineered features into a shared embedding space and applies self-attention to learn arbitrary pairwise feature interactions, isolating the contribution of attention-based feature interaction from temporal sequence modeling. Full details are in Appendix G.

4.3 Mixture of Experts

The six standalone models are combined using a learned soft Mixture-of-Experts (MoE) gating network Jacobs et al. 1991; Shazeer et al. 2017. At each trade, a three-layer MLP gate observes the current feature context, all six expert probability vectors, and causal meta-features tracking past expert reliability, producing a softmax weighting over available experts. The final prediction is a convex combination of expert distributions, trained with class-weighted NLL and entropy regularization Pereyra et al. 2017 to prevent collapse onto a single expert. Full details are in Appendix H.

4.4 Feature Engineering

4.4.1 Ticker Embeddings

Kalshi tickers such as `PRES-2024-DJT` or `KXBTCD-23DEC2024-T80000` encode asset class, resolution date, and strike. Treating them as opaque categorical IDs would prevent generalization across related markets. Instead, we map tickers to dense semantic vectors by replacing hyphens and underscores with spaces, then encoding each humanized string with the pretrained `all-MiniLM-L6-v2`

sentence transformer Reimers and Gurevych 2019, yielding an ℓ_2-normalized 384-dimensional embedding per ticker. These are reduced to $d = 8$ using UMAP McInnes et al. 2018 with cosine distance, $n_{\text{neighbors}} = 15$, and min_dist $= 0.05$.

Figure 2 shows a 2-dimensional visualization of the resulting embedding space. NFL, NBA, cryptocurrency, and equity index markets each occupy distinct regions, with contracts sharing asset class and structure sitting close together regardless of resolution date or strike. This confirms the embeddings capture both asset class and contract-type taxonomy, enabling the model to transfer dynamics across thematically related markets.

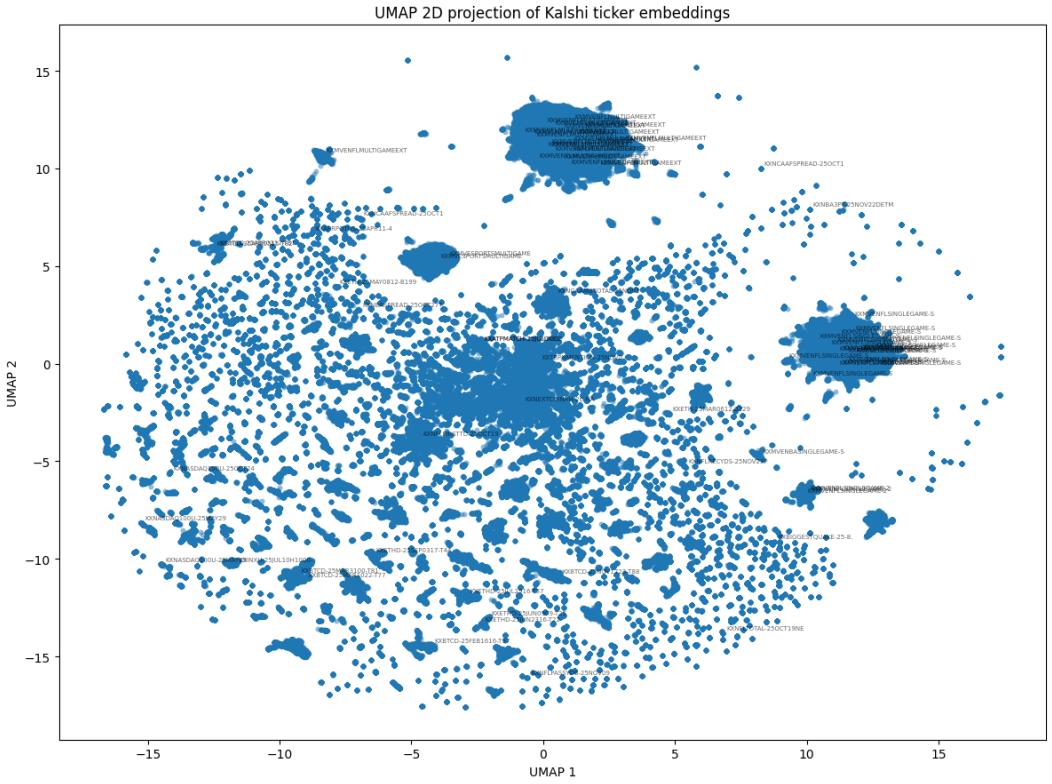

Figure 2: Two-dimensional UMAP projection of ticker embeddings. Each point is one Kalshi market; clusters correspond to thematically related contract types.

4.4.2 Standalone Model Features

We construct two complementary feature families: static semantic representations derived from ticker symbols and dynamic trade-level features capturing local price action, volume, and timing. All dynamic features are strictly causal. Table 2 summarizes all 36 model inputs.

Table 2: Model inputs. Ticker embeddings are fixed per market; all other features are causal and backward-looking.

Feature	**Dim.**	**Description**
Static (per ticker)		
`ticker_emb`$_{0..7}$	8	UMAP-reduced `all-MiniLM-L6-v2` sentence embedding of the humanized ticker symbol.
Dynamic (per trade)		
`yes_price`	1	Current yes-price p_t, in cents.
`log_volume`	1	$\log(1 + v_t)$, where v_t is the taker-side dollar volume.
`log_time_delta`	1	$\log(1 + \Delta\tau_t)$, with $\Delta\tau_t$ in seconds.
`ret_last_`k	3	Simple returns over the last $k \in \{5, 10, 20\}$ trades.
`position_in_range`	1	Relative position of p_t within the high-low range of the previous 20 trades.
`dist_from_50`	1	$\lvert p_t - 50\rvert/50$, distance from the maximum-uncertainty price.
`relative_volume`	1	v_t divided by mean volume of the previous 20 trades.
`tick`	1	Sign of the most recent price move, $\text{sign}(p_t - p_{t-1}) \in \{-1, 0, +1\}$.
`run_length`	1	Consecutive trades in the current tick direction.
`volume_since_price_change`	1	$\log(1 + \cdot)$ of cumulative volume since the last price change.
`trades_last_60s`	1	Trade count in the previous 60 seconds.
`trades_last_300s`	1	Trade count in the previous 300 seconds.
`hour_sin`, `hour_cos`	2	Cyclical encoding of hour of day.
Backward-looking (per trade, parallel to targets)		
`backward_ret_`h`m`	4	Realized signed return over the past $h \in \{5, 15, 30, 60\}$ minutes.
`backward_abs_ret_`h`m`	4	Magnitude of the corresponding backward return.
`recent_up_jumps_`h`m`	4	Count of backward up-jumps at horizon h in a trailing 30-minute window.
`recent_down_jumps_`h`m`	4	Count of backward down-jumps at horizon h in a trailing 30-minute window.
Total	**36**	

4.4.3 MoE Features

The gating network operates on meta-features derived from past expert behavior rather than raw trade data. All features are strictly causal, computed from shifted correctness indicators and past predictions only. Table 3 summarizes the full set.

Table 3: MoE meta-features, computed per horizon $h \in \{5, 15, 30, 60\}$ minutes and expert $m \in \{\text{lgbm, lstm, mamba, moirai, ftt, ctts}\}$.

Feature	**Dim.**	**Description**
Expert reliability		
`rolling_acc_`m`_w50_`h`m`	6	Rolling accuracy of expert m over the past 50 same-ticker trades. Captures local regime shifts in expert reliability.
`rolling_acc_`m`_w1000_`h`m`	6	Rolling accuracy over the past 1000 trades. Captures stable, ticker-level differences in expert skill.
Expert consensus		
`agreement_`h`m`	1	Fraction of experts predicting the modal class: $\frac{1}{M}\max_c \sum_m \mathbf{1}[\hat{y}_t^{(m,h)} = c]$, $M = 6$.
Expert availability		
`has_`m`_pred_`h`m`	6	Binary indicator for whether expert m has a valid prediction.
Total (per horizon)	**19**	
Total (all horizons)	**76**	

5 Results & Interpretation

5.1 Standalone Models

We evaluate six model families on the same temporal test split. All models are trained separately for each horizon $h \in \{5, 15, 30, 60\}$ minutes on the three-class jump classification task. Table 4 reports test-set metrics.

Table 4: Test-set performance across models and prediction horizons, with best results highlighted.

Model	Horizon	Balanced acc.	Macro F1	Weighted F1	Accuracy
LightGBM	5m	0.5260	**0.4748**	0.7402	0.7011
	15m	0.5398	**0.5003**	**0.7496**	0.7193
	30m	0.5574	**0.5272**	**0.7612**	0.7394
	60m	0.5588	**0.5380**	**0.7670**	**0.7522**
LSTM	5m	0.5163	0.4419	0.6358	0.5778
	15m	0.5352	0.4786	0.6580	0.6092
	30m	0.5336	0.4905	0.6669	0.6257
	60m	0.5513	0.5020	0.6730	0.6339
Mamba	5m	0.5099	0.4456	0.6398	0.5829
	15m	0.5287	0.4796	0.6632	0.6195
	30m	0.5406	0.5029	0.6795	0.6451
	60m	0.5469	0.5078	0.6802	0.6457
Moirai	5m	0.5280	0.4588	0.6508	0.5950
	15m	0.5480	0.4835	0.6557	0.6063
	30m	**0.5660**	0.5082	0.6685	0.6262
	60m	**0.5731**	0.5124	0.6657	0.6244
FT-Transformer	5m	**0.5399**	0.4619	0.7126	0.6565
	15m	**0.5504**	0.4826	0.7160	0.6672
	30m	0.5555	0.5035	0.7258	0.6883
	60m	0.5541	0.5176	0.7390	0.7121
CTTS	5m	0.5063	0.4813	**0.7738**	**0.7810**
	15m	0.5180	0.4943	0.7595	**0.7594**
	30m	0.5261	0.5008	0.7543	**0.7600**
	60m	0.5366	0.5128	0.7486	0.7395

All models improve with horizon, consistent with short-horizon jumps being dominated by microstructure noise. LightGBM is the strongest overall model, leading on macro F1 at every horizon and weighted F1 at all but 5 minutes, reaching 0.538 and 0.767 respectively at 60 minutes. Moirai achieves the highest balanced accuracy at longer horizons (0.573 at 60 minutes) but over-predicts directional jumps, depressing accuracy and weighted F1. CTTS leads on 5-minute accuracy and weighted F1, though its low balanced accuracy indicates majority-class bias. The FT-Transformer is competitive with all sequence models and leads on balanced accuracy at 5 and 15 minutes despite using no explicit temporal window beyond the engineered features. The overall pattern suggests that carefully engineered causal features capture most available signal, with sequence architectures offering limited incremental gains.

5.2 MoE

Table 5 reports MoE test-set performance across all four horizons.

Table 5: MoE gating network test-set performance across prediction horizons.

Horizon	Balanced acc.	Macro F1	Weighted F1	Accuracy	F1 down	F1 no jump	F1 up
5m	0.4950	0.4731	0.6995	0.6761	0.3003	0.8224	0.2967
15m	0.5132	0.4943	0.7029	0.6827	0.3368	0.8329	0.3133
30m	0.5422	0.5277	0.7199	0.7061	0.3828	0.8523	0.3480
60m	0.5477	0.5383	0.7315	0.7228	0.4033	0.8665	0.3451

At the 60-minute horizon the MoE matches LightGBM's macro F1 (0.538), slightly trails on balanced accuracy (0.548 vs. 0.559), and slightly edges weighted F1 (0.732 vs. 0.767). At the 5-minute horizon it falls below LightGBM on balanced accuracy (0.495 vs. 0.526), consistent with rolling-accuracy meta-features being unreliable when per-ticker expert histories are short.

Figure 3 summarizes the gate's average routing behavior. LightGBM receives the dominant share at every horizon (≈ 0.49–0.50, roughly $3\times$ the uniform baseline of $1/6$). The secondary expert shifts with horizon: CTTS at 5m and 15m, FT-Transformer at 30m and 60m. LSTM, Mamba, and Moirai collectively receive less than 0.15 at every horizon.

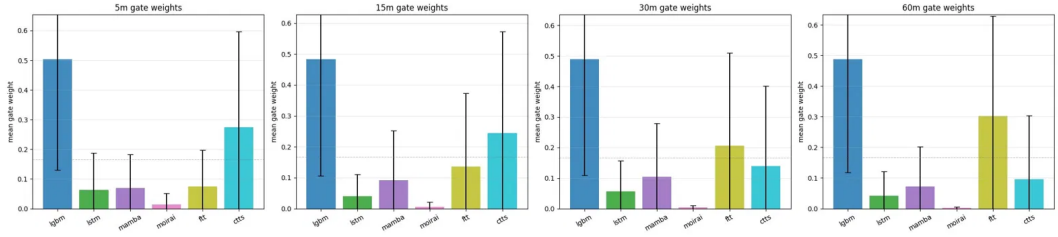

Figure 3: **Mean gate weight per expert across the test set, by horizon.** Error bars show ± 1 standard deviation; the dashed line is the uniform-prior baseline of $1/6 \approx 0.17$. LightGBM dominates at every horizon, with the secondary expert shifting from CTTS (5m, 15m) to FT-Transformer (30m, 60m).

5.3 Interpretation

What the cross-section reveals. The spread between the best and worst models on macro F1 is small (≤ 4 points) at every horizon, while the spread on weighted F1 and accuracy is much larger ($\sim$ 7–10 points). The metric most directly tied to rare-class performance is the one where models are hardest to distinguish, while the metrics dominated by no-jump prediction split the models more cleanly. The implication is that architectures differ chiefly in how confidently they identify the easy majority class, not in their ability to call jumps, which explains why ensembling produces only marginal gains.

Per-class behavior and the directional-confusion ceiling. Across all six families, directional recall sits between 0.31 and 0.49 while no-jump recall reaches 0.61–0.85. Down-vs-up flip rates are comparable in magnitude to confusion with the neutral class for every model, indicating the bottleneck is shared signal exhaustion rather than any family-specific weakness. When six independently-architected models converge to similar directional error patterns, no convex combination of them can produce different errors. Breaking through this ceiling would require either additional signal sources (orderbook depth, cross-market features, news events) or a fundamentally different problem formulation, such as the two-stage approach that separates jump detection from directional classification.

Sequence depth offers limited returns over tabular signal. LightGBM, with no access to temporal context beyond the engineered features, leads or ties on macro F1 at every horizon. This is consistent with recent tabular deep-learning literature Shwartz-Ziv and Armon 2022; Grinsztajn et al.

2022: when domain expertise has been encoded into well-chosen rolling-window features, gradient-boosted trees are difficult to beat. The engineered features (`backward_ret_*`, `recent_*_jumps_*`, `trades_last_*`) already pre-digest the temporal structure the sequence models would need to learn from raw trade data, leaving them with diminished marginal signal at $L = 32$ windows.

Comparison to the MS3 binary baseline. The MS3 milestone framed jump detection as binary, with the CNN baseline achieving F1 of 0.571, 0.703, 0.708, and 0.705 at the four horizons. Direct metric comparison with our three-class results is not meaningful given the different label spaces. The substantive advances over MS3 are the shift to directional prediction, expansion to six heterogeneous expert families, and the introduction of a learned soft gate producing interpretable expert-weight time series.

6 Conclusion & Discussion

We asked whether trade history, market identity, and microstructure features can predict the direction of large price jumps in Kalshi prediction markets. The answer is: modestly, and with important caveats. All six model families produce out-of-sample macro F1 scores in the 0.47–0.54 range, meaningfully above chance but well below the thresholds required for systematic profitable trading after transaction costs. LightGBM, operating on 36 engineered features, matches or leads every sequence architecture across horizons, reinforcing the growing evidence that well-specified tabular features are difficult for deep models to improve upon in high-frequency event-driven settings.

The MoE gating network fails to escape this ceiling. Because all six experts share the same directional confusion profile, recalling roughly one-third of jumps regardless of architecture, any convex combination inherits that weakness. The gate's learned behavior confirms this: it routes $\approx 50\%$ of weight to LightGBM and effectively ignores three of the five remaining experts.

Two limitations are worth emphasizing. First, the feature set is purely internal to Kalshi. Orderbook depth, cross-market correlations, and real-time news text are natural next additions; our results suggest they are likely necessary, not merely helpful, for pushing directional recall into actionable territory. Second, the test period shows modestly elevated volatility relative to training, which may favor or disfavor specific architectures in ways that would not generalize across market regimes.

Future work should explore a two-stage pipeline separating jump detection from directional classification, since the dominant failure mode is classifying no-jump trades as directional rather than misidentifying direction given a confirmed jump. Incorporating limit order book snapshots and cross-ticker contemporaneous signals represents the most promising path toward a structurally different error regime.

7 Broader Impact

Prediction markets serve a legitimate function: aggregating dispersed information into calibrated probability estimates that inform decision-makers in finance, policy, and media. Tools that improve price discovery efficiency can benefit this function. However, models that detect and trade ahead of price jumps, even modestly, raise distributional concerns. Gains for sophisticated algorithmic participants can come at the expense of retail traders who supply liquidity without equivalent signal access, widening informational asymmetry on the platform. Deployment at scale could also discourage retail participation, reducing the diversity of information sources that makes prediction markets socially valuable in the first place. This work is academic and does not constitute a trading system, but the same methodology applied in production would warrant careful evaluation of its net effect on market quality alongside its private profitability.

References

Becker, Jon (2026). *Prediction Market Analysis: A Framework and Dataset for Polymarket and Kalshi*. https://github.com/jon-becker/prediction-market-analysis.

Bengio, Yoshua, Patrice Simard, and Paolo Frasconi (1994). "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks* 5.2, pp. 157–166. DOI: 10.1109/72.279181. URL: https://doi.org/10.1109/72.279181.

Chen, Tianqi and Carlos Guestrin (2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, pp. 785–794. DOI: 10.1145/2939672.2939785. URL: https://arxiv.org/abs/1603.02754.

Friedman, Jerome H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine". In: *The Annals of Statistics* 29.5, pp. 1189–1232. DOI: 10.1214/aos/1013203451. URL: https://doi.org/10.1214/aos/1013203451.

Gorishniy, Yury et al. (2021). "Revisiting Deep Learning Models for Tabular Data". In: *Advances in Neural Information Processing Systems*. Vol. 34, pp. 18932–18943. URL: https://arxiv.org/abs/2106.11959.

Grinsztajn, Léo, Edouard Oyallon, and Gaël Varoquaux (2022). "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" In: *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. Vol. 35. DOI: 10.48550/arXiv.2207.08815.

Gu, Albert and Tri Dao (2023). "Mamba: Linear-Time Sequence Modeling with Selective State Spaces". In: *arXiv preprint arXiv:2312.00752*. URL: https://arxiv.org/abs/2312.00752.

Gu, Albert et al. (2020). "HiPPO: Recurrent Memory with Optimal Polynomial Projections". In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 1474–1487. URL: https://arxiv.org/abs/2008.07669.

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735. URL: https://doi.org/10.1162/neco.1997.9.8.1735.

Jacobs, Robert A. et al. (1991). "Adaptive Mixtures of Local Experts". In: *Neural Computation* 3.1, pp. 79–87. DOI: 10.1162/neco.1991.3.1.79. URL: https://www.cs.toronto.edu/~hinton/absps/jjnh91.pdf.

Jozefowicz, Rafal, Wojciech Zaremba, and Ilya Sutskever (2015). "An Empirical Exploration of Recurrent Network Architectures". In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. PMLR, pp. 2342–2350. URL: https://proceedings.mlr.press/v37/jozefowicz15.html.

Ke, Guolin et al. (2017). "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 3146–3154. URL: https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

McInnes, Leland, John Healy, and James Melville (2018). "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction". In: *arXiv preprint arXiv:1802.03426*. URL: https://arxiv.org/abs/1802.03426.

Pereyra, Gabriel et al. (2017). "Regularizing Neural Networks by Penalizing Confident Output Distributions". In: *International Conference on Learning Representations Workshop*. URL: https://arxiv.org/abs/1701.06548.

Reimers, Nils and Iryna Gurevych (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. DOI: 10.18653/v1/D19-1410. URL: https://arxiv.org/abs/1908.10855.

Rokach, Lior (2010). "Ensemble-based Classifiers". In: *Artificial Intelligence Review* 33.1, pp. 1–39. DOI: 10.1007/s10462-009-9124-7. URL: https://doi.org/10.1007/s10462-009-9124-7.

Saxe, Andrew M., James L. McClelland, and Surya Ganguli (2014). "Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1312.6120.

Shazeer, Noam et al. (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1701.06538.

Shwartz-Ziv, Ravid and Amitai Armon (2022). "Tabular Data: Deep Learning is Not All You Need". In: *Information Fusion* 81, pp. 84–90. DOI: 10.1016/j.inffus.2021.11.011.

Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. URL: https://arxiv.org/abs/1706.03762.

Woo, Gerald et al. (2024). "Unified Training of Universal Time Series Forecasting Transformers". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 53140–53164. URL: https://arxiv.org/abs/2402.02592.

Zeng, Zhen et al. (2023). "Financial Time Series Forecasting using CNN and Transformer". In: *AAAI 2023 Bridge Program on AI for Financial Services*. URL: https://arxiv.org/abs/2304.04912.

AI and Computational Tools Disclosure

Overview

This project made extensive use of large language model assistants, primarily Claude (Anthropic) and ChatGPT (OpenAI), throughout the research pipeline. All core conceptual decisions, including the jump-label framing of the forecasting problem, the multi-horizon target construction, the conditional quantile thresholding approach, the selection of model families, and the MoE gating framework, originated from the authors. AI tools were used primarily to accelerate implementation, debugging, and writing.

Algorithm Design and Architecture

High-level architectural decisions were made by the authors. These include the choice to use LightGBM as a tabular baseline, the decision to repurpose Moirai as a backbone classifier by replacing its forecasting head, the CNN-plus-Transformer combination in CTTS, the dual learning rate strategy for Moirai fine-tuning, and the MoE gating framework over expert logits. AI assistants were consulted to clarify implementation details of specific components, including the Mamba selective scan mechanism, the Moirai patchification and positional encoding interface, and the CTTS per-window normalization recipe. In these cases AI tools served as a reference layer rather than as a design agent.

Data Cleaning and Preprocessing

The feature engineering pipeline, including the definition of all 36 trade-level features, the conditional quantile thresholding for jump labels, the forward and backward return construction, and the temporal 80/20 train/test split, was designed by the authors. AI assistance was used to accelerate pandas implementation, including the `searchsorted`-based forward return computation and the vectorized per-ticker rolling feature calculations. AI tools also helped diagnose silent bugs in the `NaN`-handling logic and in correctly applying the shifted correctness indicator to prevent label leakage in the MoE meta-feature pipeline.

Exploratory Data Analysis

All EDA observations reported in the paper, including the identification of the severe right-skew in Kalshi trading activity, the zero-peaked structure of the forward return distributions, and the multimodal yes-price distribution across contract types, were made by the authors through direct inspection of the data. AI tools were used to generate boilerplate plotting code for distribution histograms, UMAP visualizations, and the return threshold overlay figures.

Code Implementation and Refactoring

AI assistance was used extensively throughout the codebase. Specific areas include the `KalshiSequenceDataset` ticker-boundary enforcement logic, the Moirai patchification and packed attention mask construction, the CTTS per-window min-max normalization, the MoE meta-feature pipeline, and the contiguous memory optimization that resolved the DataLoader worker RAM duplication issue in the Moirai training loop. In most cases, the authors specified the desired behavior and constraints, and AI tools produced initial implementations that were then reviewed, tested, and corrected by the authors.

Debugging and Error Diagnosis

AI tools were used extensively for debugging. Notable examples include diagnosing `NaN` gradient propagation under mixed precision in the Mamba and CTTS training loops, identifying the ticker-boundary contamination bug in the sequence dataset, resolving the silent RAM exhaustion caused by DataLoader forking in the Moirai pipeline, diagnosing the degenerate "always predict down" collapse in the CTTS 5-minute model under linear class weighting, and tracing shape mismatches in the Moirai patchification and pooling stages. In each case, the authors identified that an error existed; AI tools assisted in isolating root causes and proposing fixes, which the authors then validated.

Writing and Report Preparation

The report was written collaboratively by the authors with AI assistance used for editing and condensing under a strict word limit, enforcing consistent academic style across sections written by different authors, and generating LaTeX table and figure environments. All technical claims, numerical results, and interpretations are the authors' own.

Architecture Diagrams

The TikZ architecture diagrams for the LightGBM, LSTM, Mamba, Moirai, and CTTS models were produced with AI assistance. The authors specified the forward-pass structure, tensor shapes, color scheme, and layout; AI tools produced the TikZ code, which was subsequently reviewed and corrected by the authors to ensure accuracy.

Note on Tool Availability

Claude (Anthropic) was the primary AI assistant used throughout this project and was strongly preferred by the authors for its code reasoning and technical writing quality. On several occasions, Claude usage limits were reached mid-session, requiring temporary substitution with Gemini (Google). Gemini was used reluctantly and only for continuity during these interruptions, primarily for code completion and debugging tasks. The authors do not consider Gemini's contributions to have materially influenced the project's direction or outputs, and returned to Claude as soon as credits were restored.

Summary

AI tools contributed substantially to the speed and quality of implementation, debugging, and writing in this project. The research questions, modeling framework, experimental design, and interpretation of results are entirely the authors' own work. Any errors or misstatements in the report are the authors' responsibility.

A Feature Engineering

A.1 Trade-level Features

For each ticker, we sort trades chronologically and compute the following features. Let p_t denote the yes-price of trade t, v_t its taker-side dollar volume in cents, defined as

$$v_t \;=\; \begin{cases} p_t^{\text{yes}} \cdot c_t & \text{if taker\_side}_t = \text{yes} \\ p_t^{\text{no}} \cdot c_t & \text{otherwise,} \end{cases}$$

where c_t is the contract count, and τ_t the trade timestamp.

Volume. We use $\log(1 + v_t)$ as the model input to compress the heavy-tailed distribution of trade sizes.

Inter-trade time. We compute $\Delta\tau_t = \tau_t - \tau_{t-1}$ in seconds, and use $\log(1 + \Delta\tau_t)$ as a feature. This captures market activity intensity: long inter-trade gaps indicate inactive periods, while short gaps signal bursts of trading often associated with information arrival.

Short-horizon returns. We compute simple returns over the past $k \in \{5, 10, 20\}$ trades,

$$r_t^{(k)} = p_t - p_{t-k},$$

to expose recent momentum at multiple lookbacks.

Position in recent range. Over a rolling window of the last 20 trades, we compute the relative position of the current price within the local high-low range,

$$\text{pos}_t = \frac{p_t - \min_{i\in[t-20,t-1]} p_i}{\max_{i\in[t-20,t-1]} p_i - \min_{i\in[t-20,t-1]} p_i},$$

which captures whether the market is trading near recent support or resistance.

Distance from uncertainty. We compute $|p_t - 50|/50$, the normalized distance of the current yes-price from 50. This captures the market's distance from the maximum-uncertainty point and is informative for prediction-market dynamics, where prices near 0 or 100 behave very differently from prices near 50.

Relative volume. The current trade's volume is normalized by the rolling mean of the previous 20 trades' volumes, $v_t/\bar{v}_{t-20:t-1}$, isolating volume anomalies from each market's baseline activity level.

Tick direction and runs. We define the tick direction $s_t = \text{sign}(p_t - p_{t-1}) \in \{-1, 0, +1\}$ and the run length, defined recursively as the number of consecutive trades in the same nonzero direction. These features summarize short-term directional persistence.

Volume since last price change. We accumulate volume between price changes, resetting whenever the price moves, and use $\log(1 + \cdot)$ of the running total. Large accumulated volume without a price move indicates absorption of order flow and informational pressure on the resting quotes.

Rolling trade counts. We compute the number of trades occurring in the past 60 seconds and 300 seconds prior to each trade, capturing market activity at two timescales independent of trade-count windows.

Time-of-day encoding. The hour of day (with minute fraction) is encoded cyclically as $(\sin(2\pi h/24), \cos(2\pi h/24))$, preserving continuity across the day boundary.

The final feature vector concatenates the trade-level features above with the eight-dimensional ticker embedding, yielding an input representation that combines short-horizon market microstructure with a static semantic prior on the type of contract being traded.

Backward returns. For each horizon $h \in \{5, 15, 30, 60\}$ minutes, we compute a realized signed return mirroring the forward target:

$$b_t^{(h)} \;=\; p_t - p_{t^\dagger}, \qquad t^\dagger \;=\; \max\big\{i : \tau_i \le \tau_t - h,\ \text{ticker}(i) = \text{ticker}(t)\big\},$$

with $b_t^{(h)} = 0$ when no prior trade exists in the window. The absolute version $|b_t^{(h)}|$ is also exposed as a feature. These quantities provide the model with a direct, horizon-matched view of recent momentum and realized volatility, complementing the trade-count returns ($r_t^{(k)}$) which use a different timescale.

Recent jump counts. We additionally summarize how often a market has recently experienced jumps at each horizon. Concretely, for each trade t and horizon h, we count

$$\text{recent\_up\_jumps}_t^{(h)} \;=\; \big|\{i < t \;:\; \tau_i \in [\tau_t - W, \tau_t),\ b_i^{(h)} \ge \text{up\_thr}^{(h)}\}\big|,$$

and analogously for down-jumps, where $W = 30$ minutes is a fixed look-back window and the thresholds $\text{up\_thr}^{(h)}$, $\text{down\_thr}^{(h)}$ are the same train-derived values used to label the jump targets. We use a wall-clock window rather than a trade-count window so that the feature has consistent meaning regardless of market activity level. This feature exposes recent jump clustering directly to the model, capturing the regime-like behaviour in which periods of large moves tend to beget further large moves.

A.2 MoE Meta-Features

The gating network requires features that characterize the current reliability and consensus of the expert ensemble, rather than the trade-level price dynamics consumed by the standalone models. For each expert $m \in \mathcal{M} = \{\text{lgbm, lstm, mamba, moirai, ftt, ctts}\}$ and horizon $h \in \{5, 15, 30, 60\}$ minutes, we derive meta-features causally from past predictions and realized targets. Let $\hat{y}_t^{(m,h)} \in \{-1, 0, +1\}$ denote the argmax prediction of expert m at trade t for horizon h, and let $y_t^{(h)}$ denote the realized jump label.

Correctness indicators. For each expert and horizon, we define a binary correctness indicator

$$c_t^{(m,h)} \;=\; \mathbf{1}\big[\hat{y}_t^{(m,h)} = y_t^{(h)}\big].$$

To prevent leakage, all rolling aggregations are computed over the shifted series $c_{t-1}^{(m,h)}$, so the gating network observes only past prediction quality and never the correctness of the current trade.

Rolling expert accuracy. Within each ticker, we compute rolling means of the shifted correctness indicator over two windows:

$$\bar{a}_{t,w}^{(m,h)} \;=\; \frac{1}{w}\sum_{s=1}^{w} c_{t-s}^{(m,h)}, \qquad w \in \{50, 1000\},$$

with $\texttt{min\_periods} = \max(5, w/10)$ to handle short ticker histories. The short window ($w = 50$) tracks local regime shifts in expert reliability, capturing periods where a particular model has recently become systematically right or wrong on a given market. The long window ($w = 1000$) estimates stable, ticker-level differences in expert skill that persist across regimes.

Expert agreement. At each trade t and horizon h, we measure the degree of consensus among the $M = 6$ experts as the fraction predicting the modal class:

$$\text{agr}_t^{(h)} \;=\; \frac{1}{M}\max_{c \in \{-1,0,+1\}} \sum_{m \in \mathcal{M}} \mathbf{1}\big[\hat{y}_t^{(m,h)} = c\big].$$

Values near 1 indicate strong ensemble consensus; values near $1/3$ indicate maximal disagreement. The gate uses this signal to modulate confidence in any single expert's prediction.

Coverage indicators. Not all experts emit valid predictions for every trade. We define

$$\delta_t^{(m,h)} \;=\; \mathbf{1}\big[\hat{y}_t^{(m,h)} \text{ is available}\big],$$

as a binary mask used to zero out contributions from unavailable experts in the gating computation.

Final MoE representation. For each horizon h, the meta-feature vector contains $2M$ rolling accuracy features, one agreement scalar, and M coverage indicators, totalling $3M + 1 = 19$ features per horizon and $4 \times 19 = 76$ features across all horizons. This representation is concatenated with the standard trade-level features from Appendix A.1 to form the full input to the gating network.

B LightGBM Gradient-Boosted Tree Ensemble

B.1 Architecture

The LightGBM baseline consists of four independent gradient-boosted decision tree (GBDT) ensembles Ke et al. 2017, one per forecast horizon $h \in \{5, 15, 30, 60\}$ minutes. Unlike the Mamba sequence model of Appendix D, LightGBM consumes no temporal context directly: each trade is represented by the same 40-dimensional flat feature vector of Section 4.4, and all multi-trade history is condensed a priori into the engineered backward-return, recent-jump-count, and rolling-window features. Figure 4 shows the inference path for a single horizon model.

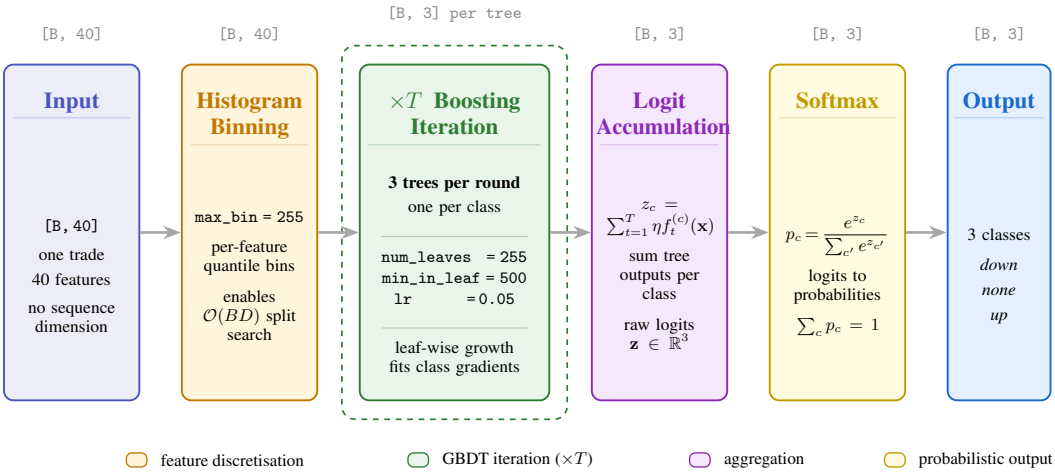

Figure 4: **LightGBM ensemble architecture for one forecast horizon.** Each trade's 40-dimensional feature vector is binned into per-feature histograms, then processed by T boosting iterations. Each iteration trains three new decision trees in parallel, one per class, using leaf-wise growth. Tree outputs accumulate into raw class logits, which a softmax converts to a three-class probability distribution. Four such ensembles are trained independently, one per horizon.

Internally, each ensemble is a sum of T boosting iterations, and at every iteration LightGBM grows three new decision trees in parallel, one per output class, to drive down the multiclass softmax cross-entropy. With `NUM_BOOST_ROUND` $= 900$, this yields up to $\approx 2{,}700$ trees per horizon model. At inference time, a trade's feature vector traces a path from root to leaf in every tree; the leaf values are accumulated separately for each class to produce three raw logits $\mathbf{z} = (z_{-1}, z_0, z_{+1})$, and a softmax converts these to class probabilities.

B.2 Why Gradient-Boosted Decision Trees

A single decision tree partitions feature space into axis-aligned rectangles and predicts a constant within each rectangle. It handles mixed-scale and missing inputs without preprocessing and learns arbitrary feature interactions, but suffers from high variance, small perturbations of the training set produce wildly different trees, and the piecewise-constant prediction surface is non-smooth in ways that hurt generalisation. Bootstrap aggregation (random forests) reduces variance by averaging T independent trees, but its parallel construction provides no mechanism for error correction: each tree solves the same problem from scratch.

Gradient boosting addresses both limitations by fitting trees *sequentially*, each one targeting the residual errors of the current ensemble Friedman 2001. Formally, given a differentiable loss $L(y, F(x))$ and an additive predictor $F^{(t)}(x) = \sum_{s=1}^{t} \eta f_s(x)$, the next tree f_{t+1} is chosen to minimise a

Table 6: LightGBM hyperparameters, identical across all four horizon models.

Hyperparameter	Value	Notes
NUM_LEAVES	255	Max leaves per tree
MIN_DATA_IN_LEAF	500	Min samples to form a leaf
MAX_BIN	255	Histogram bin count per feature
LEARNING_RATE	0.05	Shrinkage applied to each tree
FEATURE_FRACTION	0.9	Per-tree feature subsample
BAGGING_FRACTION	0.9	Per-iteration row subsample
BAGGING_FREQ	1	Resample rows every iteration
NUM_BOOST_ROUND	900	Max boosting iterations
EARLY_STOPPING	30	Validation log-loss patience
OBJECTIVE	multiclass	Softmax CE over 3 classes
NUM_CLASS	3	down / none / up

second-order Taylor expansion of the loss Chen and Guestrin 2016:

$$\mathcal{L}^{(t+1)} \approx \sum_{i=1}^{N} \left[g_i f_{t+1}(x_i) + \tfrac{1}{2} h_i f_{t+1}(x_i)^2 \right] + \Omega(f_{t+1}),$$

where $g_i = \partial_F L(y_i, F^{(t)}(x_i))$, $h_i = \partial_F^2 L(y_i, F^{(t)}(x_i))$, and Ω regularises tree complexity. For a fixed tree structure with leaves $j = 1, \ldots, J$ partitioning the data into instance sets I_j, the optimal leaf weight has a closed form $w_j^\star = -G_j/(H_j + \lambda)$ with $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$, and the corresponding loss reduction is $-\frac{1}{2} \sum_j G_j^2/(H_j + \lambda)$. Split-finding enumerates candidate splits and picks the one maximising this reduction. For multiclass classification with $K = 3$ classes, the softmax cross-entropy is differentiable per-class and the boosting iteration trains K trees jointly, with the class-k tree fitting the gradient of class-k's logit. The shrinkage factor $\eta = 0.05$ damps each tree's contribution, permitting a larger total ensemble without overfitting.

The dominant cost of vanilla GBDT is split-finding: for N instances and D features, naively scanning all candidate split points costs $\mathcal{O}(ND)$ per node, which is prohibitive at our $\sim$46M training rows. LightGBM accelerates this in two ways:

1. **Histogram-based splits.** Each feature is pre-discretised into $B = 255$ bins; split-finding then scans bins rather than raw values, reducing per-node cost to $\mathcal{O}(BD)$, roughly five orders of magnitude smaller in our setting.
2. **Leaf-wise growth.** Rather than growing the tree level by level (XGBoost's default), LightGBM at each step splits the leaf whose split yields the largest loss reduction, regardless of depth. For a fixed leaf budget this produces deeper, more asymmetric trees that capture rare interactions more effectively, at the cost of higher overfitting risk, which we control via NUM_LEAVES $= 255$, MIN_DATA_IN_LEAF $= 500$, and the stochastic regularisers FEATURE_FRACTION and BAGGING_FRACTION.

Two additional LightGBM innovations, Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB), were not required at our scale and are left at their default behaviours.

For our setting this inductive bias is well matched to the data. Kalshi features are tabular with heterogeneous scales (price in $[1, 99]$, log-volumes in $\mathbb{R}_{\geq 0}$, dimensionless ratios, periodic encodings, dense UMAP embedding coordinates). The jump targets depend on sharp non-linearities (an at-the-money contract is far more jump-prone than one near a boundary) and on cross-feature interactions (recent volatility *combined with* ticker-type embeddings, time of day, and relative volume). GBDT recovers all of this without further feature engineering, and the histogram-based implementation makes training on tens of millions of rows tractable on a single GPU.

B.3 Dataset Construction and Challenges

We reused the temporal 80/20 split, with all trades before the 80th-percentile timestamp in train and the remainder in test. Within the training partition we further carved off the most recent 10%

(chronologically) as a held-out validation fold for early-stopping diagnostics, leaving ≈ 45.9M rows for fitting, 5.1M for validation, and 12.8M for evaluation. Random k-fold splitting was avoided because it would leak future information through the engineered features: `backward_ret_60m` for a test trade can extend into the training period if trades are interleaved, and even more aggressively through the `recent_jumps` counters whose 30-minute look-back is computed from the surrounding ticker history.

The first engineering challenge was preventing label leakage through collinear or duplicate columns. We removed `no_price` (exactly $100 -$ `yes_price` since the contracts are binary), `volume` (already $\log(1+\cdot)$-transformed into `log_volume`), and obviously the four `target_price_*m`, `signed_ret_*m`, `abs_ret_*m`, and non-active `jump3_*m` columns from the feature set. The categorical `taker_side` was binarised to `taker_side_int`.

The second challenge was class imbalance. The *no jump* class accounts for 78–83% of trades depending on horizon, so a model trained with unweighted cross-entropy collapses onto the majority class. We address this by computing per-class weights from the fit-fold label counts:

$$w_c^{(h)} \;=\; \frac{N_{\text{fit}}^{(h)}}{3 \cdot N_c^{(h)}},$$

and passing them as per-row sample weights to LightGBM. The weights are computed on the fit fold only (not the full train fold) to prevent any signal from the validation split influencing model selection. LightGBM further requires non-negative integer class labels, so we remap $\{-1, 0, +1\} \to \{0, 1, 2\}$ before training and invert the mapping at evaluation.

A third issue was memory. The full feature matrix at `float32` occupies ≈ 7 GB and the fit fold alone is ≈ 5.6 GB; with the `lgb.Dataset` construction overhead and the boosting state, peak resident memory exceeds 20 GB. We mitigate this by setting `free_raw_data=True` on each `Dataset`, allowing LightGBM to release the original arrays once the histograms have been built. A faster iteration path during development was to run the entire pipeline on a 5% subsample and only commit to the full 64M-row training run once the hyperparameters had been validated.

B.4 Training Objective

Each horizon model is trained with class-weighted softmax cross-entropy on the three-class labels $y_i^{(h)} \in \{0, 1, 2\}$ (re-indexed for LightGBM):

$$\mathcal{L}^{(h)} \;=\; -\frac{1}{N} \sum_{i=1}^{N} w_{y_i^{(h)}}^{(h)} \, \log p_{y_i^{(h)}}^{(h)}(\mathbf{x}_i), \qquad p_c^{(h)}(\mathbf{x}) \;=\; \frac{\exp z_c^{(h)}(\mathbf{x})}{\sum_{c'} \exp z_{c'}^{(h)}(\mathbf{x})},$$

where $z_c^{(h)}(\mathbf{x}) = \sum_{t=1}^{T} \eta f_t^{(h,c)}(\mathbf{x})$ is the accumulated logit for class c produced by the ensemble's T class-c trees. Unlike the Mamba model of Appendix D, which exposes four horizon heads on a shared backbone and minimises the mean of the four horizon losses, the LightGBM variant trains an entirely separate ensemble per horizon and does not share parameters across h. This is partly enforced by LightGBM's API, multi-output GBDT objectives are not natively supported, and partly a deliberate choice: the four horizons exhibit different optimal feature splits and different class distributions, and independent training allows per-horizon early stopping. Validation log-loss is monitored after every boosting iteration; training halts when no improvement is observed for 30 consecutive rounds, or when `NUM_BOOST_ROUND` is reached, whichever comes first.

B.5 Feature Importance

B.6 Confusion Matrix

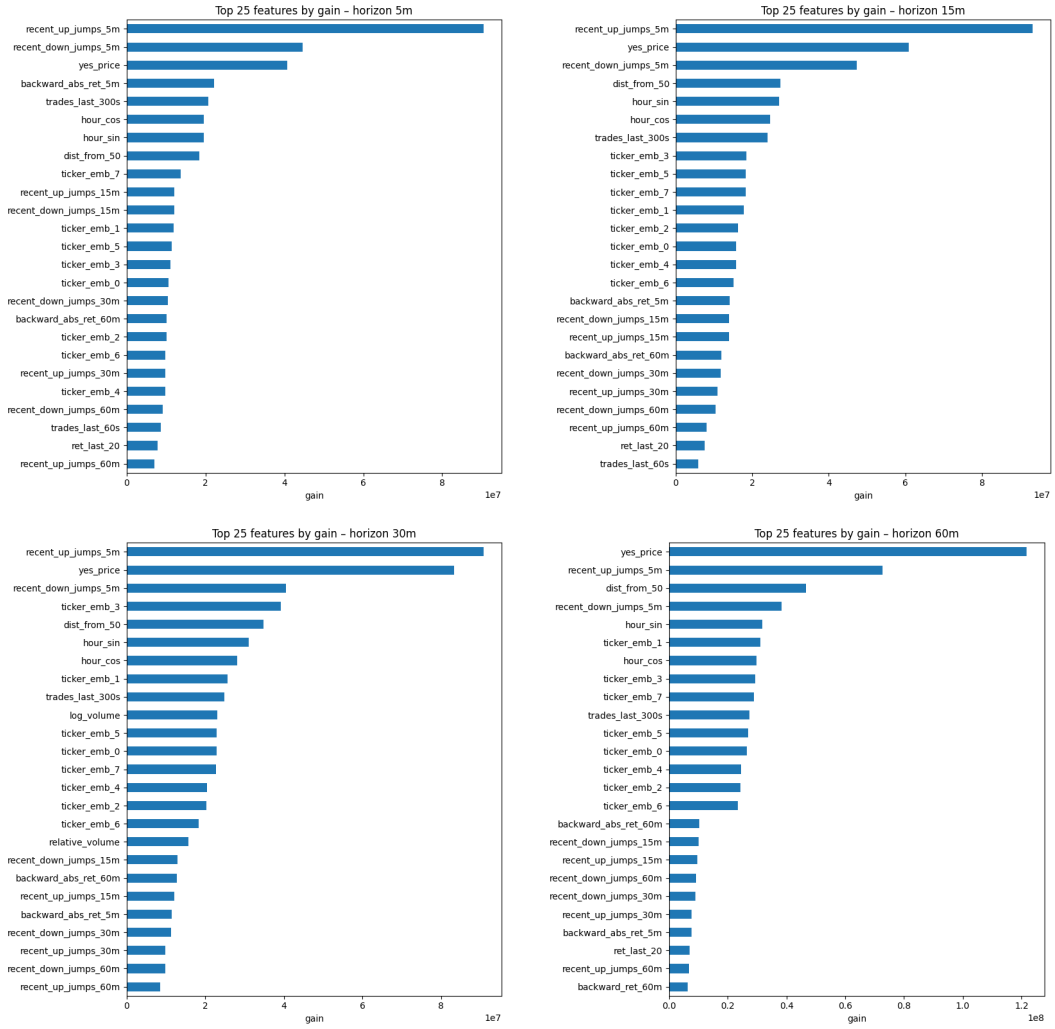

Figure 5: Top 25 features by gain for each forecast horizon (5m, 15m, 30m, 60m). The dominance of `recent_up_jumps_5m` and `recent_down_jumps_5m` across horizons, the horizon-dependent rise of `yes_price`, and the broad contribution of the eight ticker-embedding components are all visible.

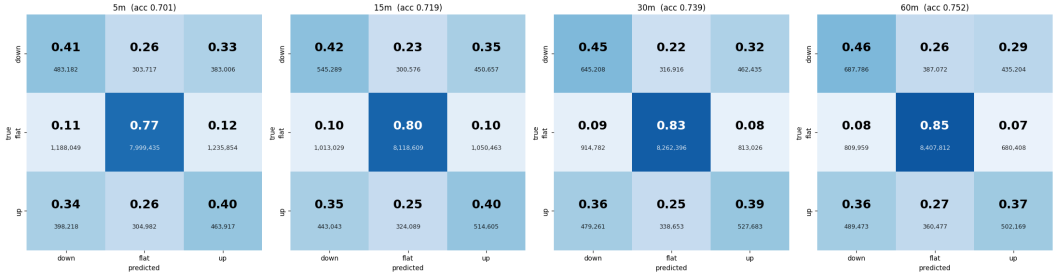

Figure 6: Confusion matrix per horizon for LightGBM model

C LSTM Sequence Classifier

C.1 Architecture

The LSTM model, `LSTMJumpPredictor`, is a unidirectional sequence classifier built on PyTorch's standard `nn.LSTM` cell Hochreiter and Schmidhuber 1997. It mirrors the input/output contract of the

Mamba model of Appendix D: it takes a window of $L = 32$ consecutive same-ticker trades, each represented by the 40-dimensional feature vector described in Section 4.4, and produces a single prediction over all four jump horizons from the final sequence position. Figure 7 gives a schematic of the full forward pass.

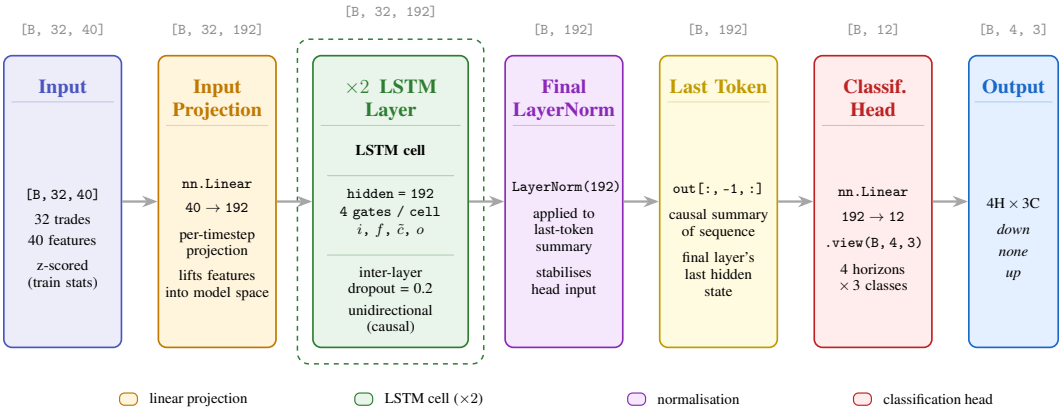

Figure 7: **LSTMJumpPredictor architecture.** A linear projection lifts the 40-dimensional per-trade feature vector into the $d_{\text{model}} = 192$ model space. Two stacked unidirectional LSTM layers with inter-layer dropout process the sequence causally. The hidden state at the final timestep is extracted, layer-normalised, and passed through a linear head producing logits over all four jump horizons simultaneously.

The input projection is a single $\texttt{nn.Linear}(40, 192)$ applied pointwise across the time dimension. Each projected vector is then consumed by a two-layer unidirectional LSTM with hidden size 192 and inter-layer dropout of 0.2. After the recurrent stack, the output at the final timestep $\mathbf{x}_{:,-1,:} \in \mathbb{R}^{B\times 192}$ is extracted as the causal summary of the window, passed through $\texttt{LayerNorm}(192)$ and a dropout layer, and projected by a single linear head $\texttt{nn.Linear}(192, 12)$ to a tensor of shape $[B, 4, 3]$, one logit triple per horizon. The recurrent weights $\mathbf{W}_{ih}$ are initialised with Xavier uniform, $\mathbf{W}_{hh}$ with orthogonal initialisation Saxe et al. 2014 for gradient stability, and the forget-gate bias is initialised to 1 following the recommendation of Jozefowicz et al. Jozefowicz et al. 2015, which biases the cell toward retaining information early in training.

Table 7: LSTM model hyperparameters.

Hyperparameter	Value	Notes
HIDDEN_SIZE	192	LSTM hidden / model dimension
NUM_LAYERS	2	Stacked LSTM layers
DROPOUT	0.2	Inter-layer + head dropout
DIRECTIONAL	uni	Strictly causal
SEQ_LEN	32	Trades per window
STRIDE	16	Step between windows
LR	3e-4	AdamW learning rate
MAX_EPOCHS	30	Upper bound on training
PATIENCE	3	Epochs of no val improvement
BATCH_SIZE	256	Mini-batch size
WARMUP_STEPS	500	Linear warmup then cosine decay
GRAD_CLIP	1.0	Max gradient norm

C.2 Why Long Short-Term Memory Networks

The vanilla RNN recurrence $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t)$ suffers from a well-known pathology: the gradient of the loss with respect to early-timestep parameters is a product of T Jacobians, which either vanishes or explodes exponentially in T Bengio et al. 1994. In practice, vanilla RNNs cannot

reliably propagate information beyond roughly 10 timesteps, which is shorter than the windows we need to process even at $L = 32$.

The Long Short-Term Memory cell Hochreiter and Schmidhuber 1997 solves this by introducing an additive cell state $\mathbf{c}_t$ controlled by three multiplicative gates:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad \text{(input gate)} \tag{1}$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad \text{(forget gate)} \tag{2}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad \text{(output gate)} \tag{3}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad \text{(candidate cell)} \tag{4}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{(cell update)} \tag{5}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(hidden output)} \tag{6}$$

The critical structural property is the cell-state recurrence $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$. When the forget gate $\mathbf{f}_t$ saturates near 1, the cell state passes through nearly unchanged, and the backward gradient $\partial \mathbf{c}_T / \partial \mathbf{c}_t$ is close to the identity rather than a product of contractive Jacobians. This is the same constant-error-carousel mechanism that motivated the original LSTM design, and it allows gradient signal to flow over hundreds of timesteps when the gates learn to remain open.

Conceptually, the LSTM and the selective state-space model of Appendix D occupy adjacent points in design space: both maintain a recurrent state, both use input-dependent gates to decide what to write into and read out of that state, and both train by backpropagation through time. The differences are mechanical rather than conceptual. The LSTM applies four independent gate networks per timestep and processes the sequence strictly serially, since each gate depends on the previous hidden state; Mamba constrains its recurrence to be linear in the hidden state, which enables a parallel scan and removes the sequential bottleneck during training Gu and Dao 2023. For windows of length $L = 32$ the parallelism advantage is modest, and the LSTM serves as a strong, well-understood baseline for our setting.

A property worth noting is that the LSTM was not designed with the inductive biases that the input data exhibits. Like Mamba it can learn to ignore quiet stretches of trades, but its forget gate is bounded in $(0, 1)$ and trained from a generic initialisation, whereas Mamba's HiPPO-initialised state matrix imposes a continuous-time memory prior Gu et al. 2020 that is provably optimal for compressing long histories. We therefore expect the LSTM to be competitive at short sequence lengths but to scale less gracefully with L than Mamba; we did not test this directly because both models were evaluated at the same fixed $L = 32$.

C.3 Dataset Construction and Challenges

We reused the `KalshiSequenceDataset` pipeline from Appendix D unchanged. Windows of length $L = 32$ are generated independently within each ticker group, with stride $S = 16$, and any window containing a `NaN` or `inf` feature is discarded entirely; the early rows of each ticker frequently have undefined lag features for which imputation would constitute look-ahead, so dropping is preferred to filling. The same temporal 80/20 split established in preprocessing is used to assign windows to train and test partitions. To monitor for early stopping, we further carved the last 10% of training windows (by timestamp of the prediction trade) into a held-out validation fold, leaving approximately 90% of training windows for gradient updates and 10% for early stopping.

Feature standardisation was performed once at dataset construction time using the per-feature mean and standard deviation computed on the fit fold of the training partition. The same statistics were applied verbatim to the validation and test partitions; no test-set information enters the normalisation. Constant features (which would yield $\sigma = 0$) had their standard deviation replaced by 1 before division.

Class imbalance was handled identically to the Mamba and LightGBM variants. No-jump trades constitute 78–83% of all windows depending on horizon, so we computed per-class weights from the fit-fold label counts:

$$w_c^{(h)} = \frac{N_{\text{fit}}^{(h)}}{3 \cdot N_c^{(h)}},$$

and passed them to a separate `nn.CrossEntropyLoss` instance for each of the four horizon heads. The overall training loss is the mean of the four horizon-specific losses, identical to Mamba's formulation.

A specific issue we encountered with the LSTM that did not appear in the Mamba variant was sensitivity to exploding gradients early in training. Without gradient clipping, the loss on the first ~ 50 batches was unstable and occasionally produced `NaN` values. We addressed this by clipping the global gradient norm to 1.0 after the backward pass via `torch.nn.utils.clip_grad_norm_`, which we found sufficient to stabilise training across all seeds tested. We left automatic mixed precision disabled for the LSTM: PyTorch's cuDNN LSTM kernel has known numerical-stability issues in `float16`, and the speed gain on recurrent models is smaller than for feed-forward architectures.

C.4 Training Objective

The model is trained with class-weighted cross-entropy on the three-class labels $y_t^{(h)} \in \{-1, 0, +1\}$ for all four horizons simultaneously. For a batch of windows $\{(\mathbf{X}_i, \mathbf{y}_i)\}$ where $\mathbf{y}_i \in \{0, 1, 2\}^4$ (re-indexed for PyTorch), the loss is:

$$\mathcal{L} = \frac{1}{4} \sum_{h=1}^{4} \mathcal{L}_{\text{CE}}^{(h)} \left(\hat{\mathbf{y}}^{(h)},\, \mathbf{y}^{(h)} \right), \qquad \hat{\mathbf{y}}^{(h)} = \texttt{head}(\mathbf{x}_{:,-1,:})[:,\, h, :].$$

This is identical in form to the Mamba objective and to the per-horizon LightGBM objectives of Appendix B, modulo the LightGBM variant's choice to train independent ensembles per horizon rather than a shared backbone. Training proceeds with AdamW ($\beta = (0.9, 0.999)$, weight decay 0.01), a linear warmup over 500 steps followed by cosine decay to zero over the remaining budget, and a maximum of 30 epochs. Validation loss is computed after each epoch on the held-out 10% fold; training halts when no improvement is observed for 3 consecutive epochs, at which point the checkpoint corresponding to the lowest observed validation loss is restored for downstream evaluation.

C.5 Feature Importance & Confusion Matrix

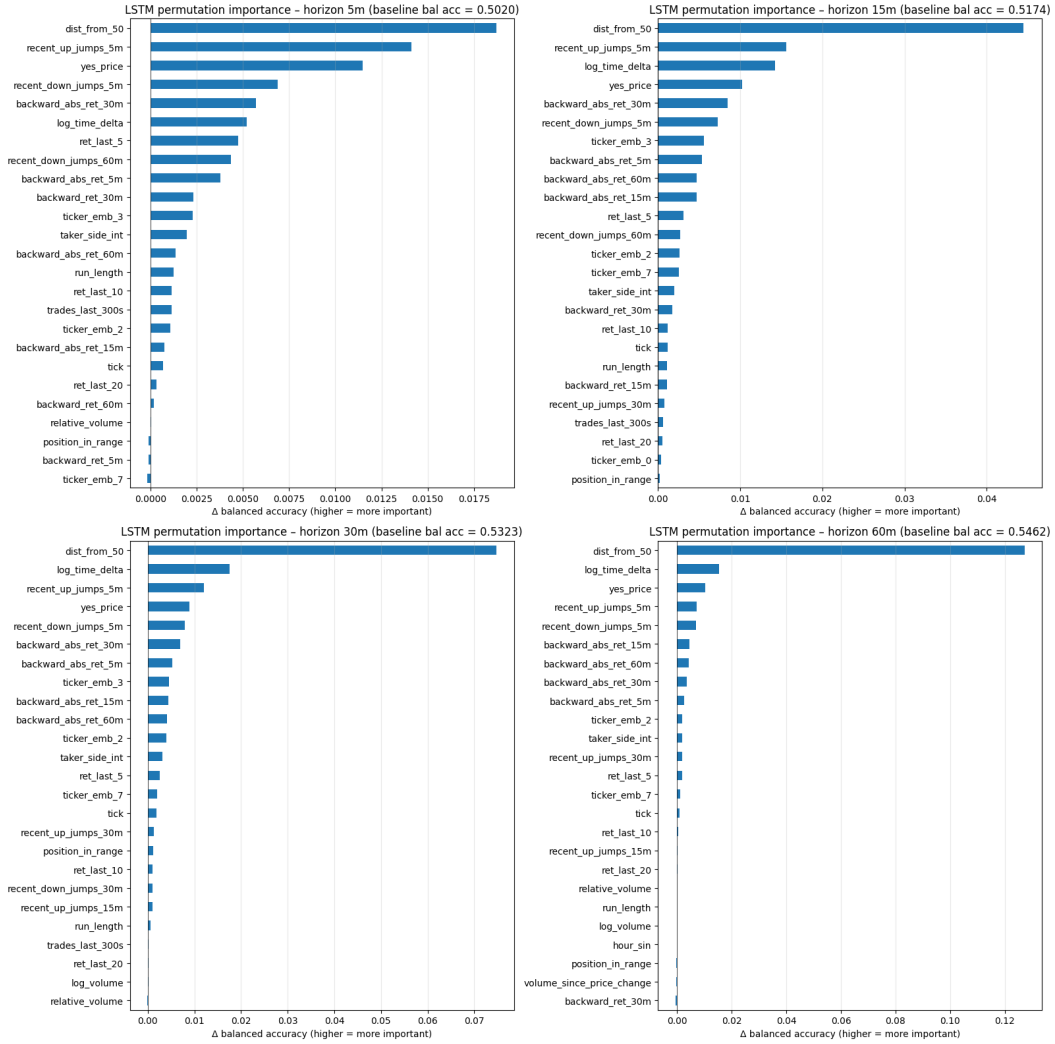

Figure 8: LSTM Permutation Importance for top 25 features

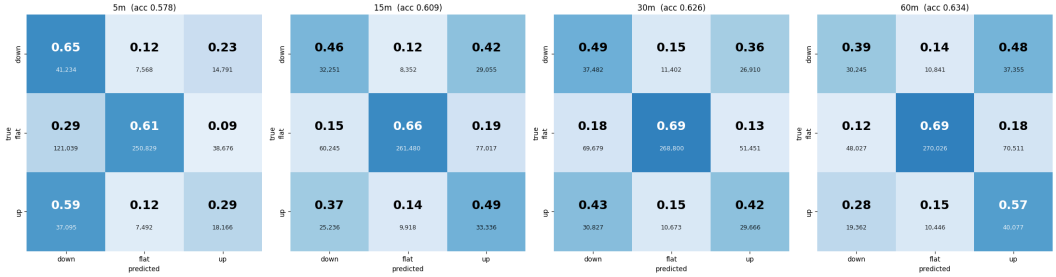

Figure 9: Confusion Matrix per horizon for LSTM model

D Mamba Sequence Classifier

D.1 Architecture

The Mamba model, `MambaJumpPredictor`, is a purely causal sequence classifier built on the Mamba selective state-space block Gu and Dao 2023. It takes as input a window of $L = 32$ consecutive same-ticker trades, each represented by the 40-dimensional feature vector described in Section 4.4, and

produces a single prediction over all four jump horizons from the final sequence position. Figure 10 gives a schematic of the full forward pass.

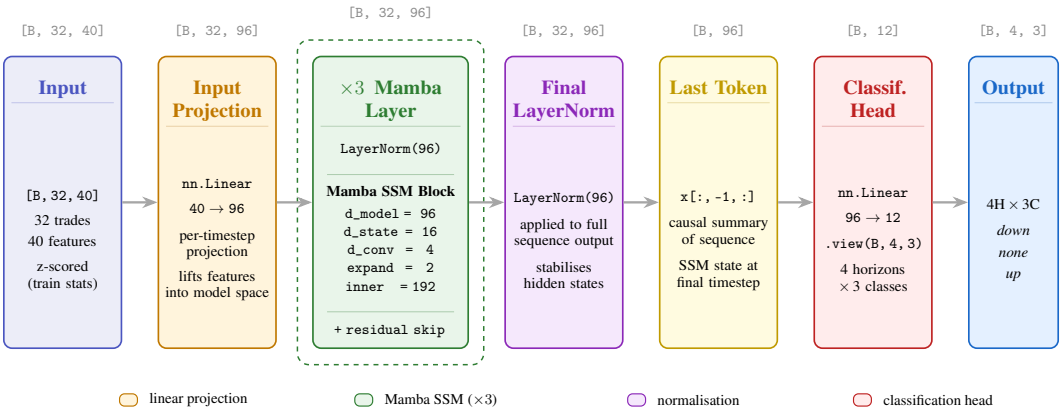

Figure 10: **MambaJumpPredictor architecture.** A linear projection lifts the 40-dimensional per-trade feature vector into the $d_{\text{model}} = 96$ model space. Three Mamba SSM layers with pre-norm and residual connections process the sequence causally. The hidden state at the final timestep is extracted and passed through a linear head producing logits over all four jump horizons simultaneously.

The input projection is a single $\texttt{nn.Linear}(40, 96)$ applied pointwise across the time dimension, lifting each trade's feature vector into the model's $d_{\text{model}} = 96$ space. Each of the $N = 3$ subsequent layers applies a pre-norm pattern: the residual stream is first passed through a $\texttt{LayerNorm}(96)$, then through the Mamba SSM block, and the result is added back to the residual. The Mamba block is parametrized with $d_{\text{state}} = 16$, $d_{\text{conv}} = 4$, and expand $= 2$, meaning the block widens the representation to an inner dimension of $2 \times 96 = 192$ before projecting back. After all three layers, a final $\texttt{LayerNorm}(96)$ is applied to the full sequence output, and the hidden state at the last position $\mathbf{x}_{:,-1,:} \in \mathbb{R}^{B \times 96}$ is extracted as the causal summary of the window. A single linear head $\texttt{nn.Linear}(96, 12)$ maps this to a tensor of shape $[B, 4, 3]$, one logit triple per horizon.

Table 8: Mamba model hyperparameters.

Hyperparameter	Value	Notes
D_MODEL	96	Model dimension
D_STATE	16	SSM state dimension
D_CONV	4	Depthwise conv kernel size
EXPAND	2	Inner-dim expansion factor
N_LAYERS	3	Number of Mamba layers
SEQ_LEN	32	Trades per window
STRIDE	16	Step between windows
LR	3e-4	AdamW learning rate
EPOCHS	10	Training epochs
BATCH_SIZE	256	Mini-batch size
WARMUP_STEPS	500	Linear warmup then cosine decay

D.2 Why Selective State Space Models

A standard RNN maintains a hidden state $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t)$, which suffers from vanishing gradients over long sequences, cannot be parallelized during training, and relies on arbitrary learned matrices with no structural prior. State Space Models (SSMs) address the first two problems by grounding the recurrence in continuous-time control theory:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \qquad y(t) = \mathbf{C}\mathbf{h}(t),$$

where $\mathbf{A}$ is initialized via HiPPO Gu et al. 2020 to be mathematically optimal for history compression and has eigenvalue constraints that stabilize gradients by design rather than luck. When $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ are

fixed across timesteps, the recurrence unrolls into a convolution computable in $\mathcal{O}(n \log n)$ via FFT, removing the sequential bottleneck entirely.

The limitation of fixed SSMs is that the same filter slides over every position regardless of content, the model cannot selectively attend to informative inputs over noise. Mamba Gu and Dao 2023 resolves this by making $\mathbf{B}$, $\mathbf{C}$, and the discretization step Δ functions of the current input:

$$\mathbf{h}_t = \mathbf{A}(\Delta_t)\,\mathbf{h}_{t-1} + \mathbf{B}(\mathbf{x}_t)\,\mathbf{x}_t, \qquad y_t = \mathbf{C}(\mathbf{x}_t)\,\mathbf{h}_t.$$

Intuitively, $\Delta(\mathbf{x}_t)$ controls how long a token's influence persists in memory, $\mathbf{B}(\mathbf{x}_t)$ governs what to write into the state, and $\mathbf{C}(\mathbf{x}_t)$ determines what to read out. Input-dependence invalidates the FFT convolution trick since the kernel now varies at every position. Mamba replaces it with a *parallel scan*: the recurrence is strictly linear, so consecutive steps can be combined associatively and reduced in a binary tree of $\mathcal{O}(\log n)$ depth, implemented as a custom CUDA kernel.

For our setting this inductive bias is well matched to the data. Kalshi trade sequences consist of long stretches of uninformative noise punctuated by sharp, information-rich price moves. A selective gate can learn to compress quiet periods into its state cheaply while updating aggressively on genuine shocks, precisely the behaviour required for jump prediction.

D.3 Dataset Construction and Challenges

The primary engineering challenge was constructing a sequence dataset that strictly respected ticker boundaries. Because our trades are interleaved across hundreds of markets, naïvely sliding a window of length 32 over the full sorted dataframe would routinely produce windows whose first and last trades belong to different tickers, mixing up the price dynamics of entirely unrelated contracts. We resolved this in `KalshiSequenceDataset` by grouping trades by ticker, sorting each group chronologically, and independently generating windows within each group. Windows whose 32-trade span contains any `NaN` or `inf` feature value are discarded entirely rather than imputed: the early rows of each per-ticker group frequently have undefined lag features (e.g., there is no previous trade for $r_t^{(k)}$ when $t < k$), and filling them would constitute a form of look-ahead within the feature computation.

A second challenge arose from class imbalance. No-jump trades constitute roughly 80–83% of all windows (Table 1), so a model trained with unweighted cross-entropy collapses onto the majority class. We address this by computing per-class weights from the training label counts:

$$w_c^{(h)} \;=\; \frac{N_{\text{total}}^{(h)}}{3 \cdot N_c^{(h)}},$$

normalized so that the weights average to 1 within each horizon, and passing them to a separate `nn.CrossEntropyLoss` instance for each of the four horizon heads. The overall training loss is the mean of the four horizon-specific losses.

A third issue was numerical stability under mixed precision. Mamba's selective scan involves exponentiation and gating operations that can produce `NaN` gradients in `float16` early in training, before the model's internal statistics stabilize. We kept automatic mixed precision (`USE_AMP`) disabled until the loss was confirmed finite on several consecutive batches, then enabled it for speed. The AdamW optimizer with a 500-step linear warmup followed by cosine decay to zero was sufficient to keep training stable.

D.4 Training Objective

The model is trained with class-weighted cross-entropy on the three-class labels $y_t^{(h)} \in \{-1, 0, +1\}$ for all four horizons simultaneously. For a batch of windows $\{(\mathbf{X}_i, \mathbf{y}_i)\}$ where $\mathbf{y}_i \in \{0, 1, 2\}^4$ (re-indexed for PyTorch), the loss is:

$$\mathcal{L} = \frac{1}{4} \sum_{h=1}^{4} \mathcal{L}_{\text{CE}}^{(h)} \left(\hat{\mathbf{y}}^{(h)},\, \mathbf{y}^{(h)} \right), \qquad \hat{\mathbf{y}}^{(h)} = \texttt{head}(\mathbf{x}_{:,-1,:})[:,\, h, :].$$

No auxiliary regression objective is used in the Mamba variant; the classification signal alone is sufficient given the model's parameter budget.

E Moirai Backbone Classifier

E.1 Architecture

The Moirai model, `MoiraiBackboneClassifier`, repurposes the pretrained `Salesforce/moirai-1.1-R-small` time-series foundation model Woo et al. 2024 as a sequence encoder and attaches a lightweight multi-horizon classification head in place of its native forecasting distribution. Figure 11 provides a schematic of the forward pass.

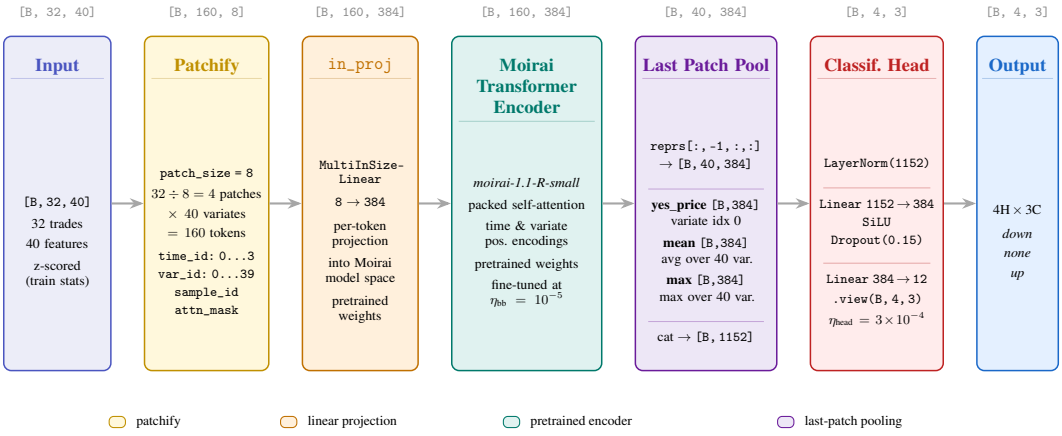

Figure 11: **MoiraiBackboneClassifier architecture.** The input sequence is patchified into 160 tokens and projected into the Moirai model space ($d = 384$) by the pretrained `in_proj` layer. The Moirai Transformer encoder processes all tokens with packed self-attention and time/variate positional encodings. Three pooled views of the final patch are concatenated into a 1152-dimensional vector and passed through a four-step MLP head.

Replacing the forecasting head. Moirai was designed for probabilistic time-series forecasting: given a context window of observed values, it predicts a parametric distribution over the next H steps in the series index. In our setting the "next steps" would be the next trades, not the future minutes over which our jump labels are defined, making the native forecasting head semantically misaligned with our targets. We therefore extract only the encoder representations and discard the distribution head entirely. The backbone is treated as a feature extractor from which we read out the final patch's hidden states; the four jump-horizon labels are then predicted by a separately initialized MLP head trained on top of those frozen or fine-tuned representations.

Patchification. Moirai operates on patched multivariate time series. Given an input $\mathbf{X} \in \mathbb{R}^{B \times L \times F}$ with $L = 32$ timesteps and $F = 40$ features, we reshape into non-overlapping patches of size $P = 8$:

$$\mathbf{X} \longrightarrow \mathbb{R}^{B \times (L/P) \times F \times P} = \mathbb{R}^{B \times 4 \times 40 \times 8},$$

then permute and flatten across the patch and feature axes to produce $n_{\text{tokens}} = (L/P) \times F = 4 \times 40 = 160$ tokens per sequence, each of dimension $P = 8$. Each token corresponds to one patch of one variate. The requirement that P divides L and belongs to the set of patch sizes supported by the checkpoint (verified at instantiation) constrains the patchification grid; $P = 8, L = 32$ satisfies both.

Positional encoding. Each of the 160 tokens receives three structured IDs injected alongside the patch content: a `time_id` $\in \{0, 1, 2, 3\}$ indexing which of the four patches it belongs to, a `variate_id` $\in \{0, \ldots, 39\}$ identifying the feature channel, and a `sample_id` for batch-level packed attention masking. The `packed_attention_mask` is constructed so that tokens from different sequences in the batch do not attend to each other. These IDs are passed directly to Moirai's Transformer encoder, which uses them to compute rotary and variate-specific position encodings. We do not add a separate classification token; instead we extract representations from the last patch position after encoding.

Encoder. The 160 tokens (each of dimension $P = 8$) are first mapped to the model dimension $d_{\text{model}} = 384$ by Moirai's `in_proj` (`MultiInSizeLinear`), which supports variable patch sizes via a set of learned weight matrices indexed by P. The resulting $[B, 160, 384]$ tensor is then processed by the Moirai Transformer encoder with full (non-causal) packed attention across time and variate tokens within each sequence. The encoder outputs a tensor of the same shape.

Pooling and classification head. After encoding, we reshape the token representations back to $[B, n_{\text{patches}}, F, d_{\text{model}}] = [B, 4, 40, 384]$ and extract the last patch slice $\mathbf{Z} = \text{reprs}_{:,-1,:,:} \in \mathbb{R}^{B\times 40\times 384}$. We then construct three pooled views of this slice:

$$\begin{aligned}
\mathbf{z}^{\text{price}} &= \mathbf{Z}_{:,0,:} \in \mathbb{R}^{B\times 384}, \quad \text{(variate 0 = yes_price)} \\
\mathbf{z}^{\text{mean}} &= \tfrac{1}{40}\textstyle\sum_{f=0}^{39} \mathbf{Z}_{:,f,:} \in \mathbb{R}^{B\times 384}, \\
\mathbf{z}^{\text{max}} &= \max_{f} \mathbf{Z}_{:,f,:} \in \mathbb{R}^{B\times 384}.
\end{aligned}$$

These three vectors are concatenated to form a $3 \times 384 = 1152$-dimensional representation, which is passed through the classification head:

$$\text{Head}: \quad \mathbb{R}^{1152} \xrightarrow{\text{LN}} \mathbb{R}^{1152} \xrightarrow{\text{Linear}} \mathbb{R}^{384} \xrightarrow{\text{SiLU}} \xrightarrow{\text{Drop}(0.15)} \mathbb{R}^{384} \xrightarrow{\text{Linear}} \mathbb{R}^{12} \xrightarrow{\text{view}} \mathbb{R}^{4\times 3}.$$

Table 9: Moirai classifier hyperparameters.

Hyperparameter	Value	Notes
`MOIRAI_MODEL_ID`	`moirai-1.1-R-small`	Salesforce pretrained checkpoint
`D_MODEL`	384	Moirai encoder width
`PATCH_SIZE`	8	Trades per patch token
`N_PATCHES`	4	$L/P = 32/8$
`N_TOKENS`	160	$N_{\text{patches}} \times F = 4 \times 40$
`SEQ_LEN`	32	Trades per window
`STRIDE`	16	Step between windows
`LR_BACKBONE`	1e-5	Fine-tuning rate for encoder
`LR_HEAD`	3e-4	Learning rate for classification head
`WEIGHT_DECAY`	1e-2	AdamW weight decay
`EPOCHS`	3	Training epochs
`BATCH_SIZE`	256	Mini-batch size
`WARMUP_STEPS`	500	Linear warmup
`USE_AMP`	True	Required for practical GPU throughput

E.2 Dataset Vectorization and Memory Challenges

The Moirai dataset implementation departed significantly from the PyTorch `DataLoader`-based approach used in Mamba, due to a severe memory duplication problem that emerged at scale.

In the Mamba pipeline, we stored per-ticker feature arrays as Python lists of NumPy arrays inside the `Dataset` object. When `DataLoader` spawns worker processes (with `num_workers` > 0) using Python's `fork`-based multiprocessing, each worker receives a copy of the entire dataset object. With 60+ million trades split into hundreds of per-ticker arrays, each worker process duplicated several gigabytes of feature data, exhausting Colab's RAM before a single batch was served and causing silent stalls rather than explicit out-of-memory errors.

We resolved this by replacing the ticker-grouped storage with a single contiguous NumPy matrix $\mathbf{X} \in \mathbb{R}^{N\times 40}$ covering all trades in sorted ticker/time order, and pre-computing integer start indices for all valid windows across all tickers in a vectorized pass over the group boundaries. Window retrieval then becomes a simple gather:

$$\mathbf{X}_{\text{batch}}[i] = \mathbf{X}[\texttt{starts}[i] \,:\, \texttt{starts}[i] + L],$$

which avoids any object-level copying and keeps RAM usage roughly proportional to the dataset size rather than $N_{\text{workers}}\times$ dataset size. We disabled `DataLoader` workers entirely (`NUM_WORKERS`

$= 0$) and replaced them with a lightweight custom batch iterator that shuffles a flat index array and calls `numpy` fancy indexing on the preallocated matrix. This approach reduced peak RAM by approximately $3\times$ and eliminated the stalling behaviour.

A second challenge specific to Moirai was the presence of irregular, event-time trade data. Moirai was pretrained on resampled, regularly spaced time series with a meaningful frequency metadata field. Our trades arrive at irregular intervals (inter-trade gaps range from milliseconds during bursts to hours during quiet periods), so the `prediction_length` argument, which in normal Moirai usage indicates how many regularly spaced future steps to forecast, has no natural interpretation: future trades do not correspond to future minutes. We sidestepped this mismatch entirely by not invoking the forecasting distribution head at all, calling only the encoder and reading off internal representations. The `time_id` passed to the encoder indexes patch position (0–3) rather than wall-clock time, which is a valid use of Moirai's positional encoding as long as it is consistent across train and inference.

E.3 Dual Learning Rates

Because the backbone carries pretrained representations that should not be destroyed in the first training steps, we use separate parameter groups with a $30\times$ lower learning rate for the encoder than for the classification head:

$$\eta_{\text{backbone}} = 10^{-5}, \qquad \eta_{\text{head}} = 3 \times 10^{-4}.$$

This is implemented by passing two parameter groups to AdamW with the same weight decay $\lambda = 10^{-2}$ but different learning rates. Optionally, the backbone can be frozen entirely (`FREEZE_BACKBONE = True`), in which case only the 1.5M head parameters are updated; in practice, light fine-tuning of the backbone improved validation macro-F1 by approximately 1–2 percentage points across horizons compared to a frozen backbone.

F CTTS: CNN-Tokenized Transformer Sequence Classifier

F.1 Architecture

The CTTS model, `CTTSJumpPredictor`, follows the CNN-and-Transformer hybrid architecture of Zeng et al. Zeng et al. 2023, in which a 1D convolutional stack acts as a learnable tokenizer over a time series and a Transformer encoder operates on the resulting token sequence. Relative to the Mamba and LSTM sequence classifiers of Appendices D and C, CTTS uses a deeper input window ($L = 64$ trades versus 32), explicit *learnable* tokens (produced by the CNN rather than read off raw trades), and full bidirectional self-attention over the tokens; relative to the FT-Transformer it operates on a temporal sequence rather than a flat per-trade feature vector. Unlike the other sequence models in this paper, CTTS is trained as one independent model per horizon rather than as a shared backbone with multiple heads, following the per-horizon-LightGBM convention in Appendix B.

The forward pass proceeds in five stages, summarised in Figure 12. First, a window of $L = 64$ same-ticker trades, each represented by the 40-dimensional feature vector from Section 4.4, is z-scored using per-feature training-set statistics. The `yes_price` channel additionally receives a per-window min-max rescaling to $[0, 1]$, following the original CTTS preprocessing recipe; this normalisation is computed inside the dataset `__getitem__` and not from training statistics. Second, a 3-block 1D CNN tokenizer convolves the resulting input $\mathbf{X} \in \mathbb{R}^{B\times 40\times 64}$ into a token sequence $\mathbf{Z} \in \mathbb{R}^{B\times 16\times 256}$. Each block applies two depth-preserving Conv1d layers with kernel 3, BatchNorm, and ReLU, followed by max-pooling with stride 2 to halve the sequence length; the channel widths grow $40 \to 64 \to 128 \to 256$ across the three blocks and the spatial dimension shrinks $64 \to 32 \to 16$. Third, a learnable `[CLS]` token is prepended to the token sequence and a sinusoidal positional encoding is added to all 17 positions. Fourth, a 4-layer Transformer encoder with $d_{\text{model}} = 256$, 4 attention heads, $d_{\text{ff}} = 512$, pre-LayerNorm, GELU activations, and dropout 0.3 processes the augmented token sequence. Fifth, the `[CLS]` representation is passed through $\texttt{LayerNorm}(256)$ and a two-layer MLP head ($256 \to 128 \to 3$) producing three logits over the $\{-1, 0, +1\}$ classes. The total parameter count is approximately 2.3M per horizon, comparable to the Mamba model and an order of magnitude smaller than the Moirai backbone.

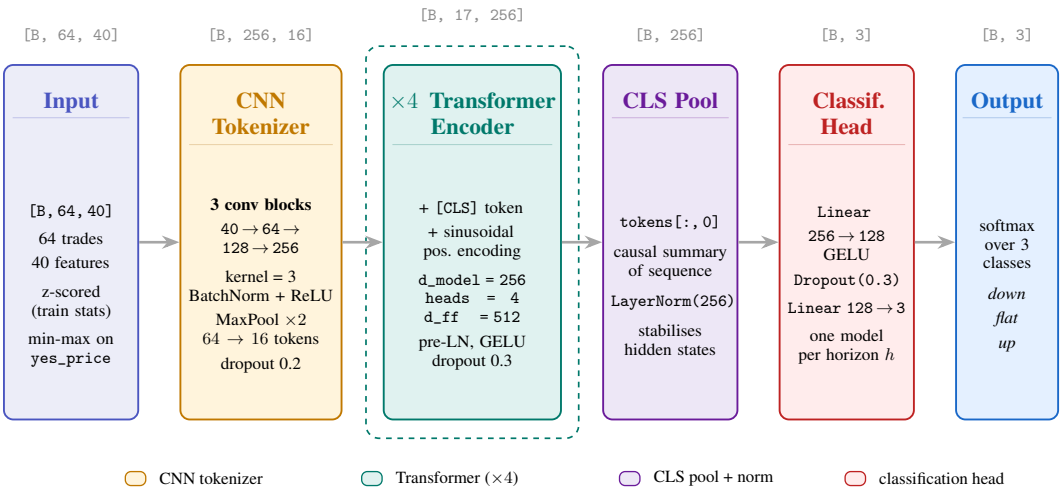

Figure 12: **CTTSJumpPredictor architecture for one forecast horizon.** A window of 64 trades × 40 features is z-scored using training-set statistics; `yes_price` additionally receives a per-window min-max rescaling. The CNN tokenizer compresses the input to 16 tokens of dimension 256. A learnable `[CLS]` token is prepended and sinusoidal positional encodings are added before a 4-layer Transformer encoder processes the 17-token sequence. The `[CLS]` representation is layer-normalised and passed through a two-layer MLP head producing three softmax logits. Four such models are trained independently, one per horizon.

Table 10: CTTS model hyperparameters, identical across all four horizon models.

Hyperparameter	Value	Notes
LOOKBACK	64	Trades per window
CNN_CHANNELS	64, 128, 256	Per-block channel widths
CNN_KERNEL	3	Conv1d kernel size
CNN_DROPOUT	0.2	Inter-block dropout
TF_DEPTH	4	Transformer encoder layers
TF_HEADS	4	Self-attention heads
TF_DIM_FF	512	Feedforward inner dimension
TF_DROPOUT	0.3	Transformer + head dropout
POOL	cls	Read [CLS] after encoder
LR	3e-4	AdamW learning rate
WEIGHT_DECAY	1e-4	AdamW weight decay
MAX_EPOCHS	25	Upper bound on training
PATIENCE	3	Epochs of no val improvement
BATCH_SIZE	256	Mini-batch size
GRAD_CLIP	1.0	Max gradient norm
CLASS_WEIGHT_POWER	0.5	sqrt-inverse-frequency weighting
USE_AMP	False	Disabled following NaN-loss at 5m

F.2 Why a CNN Tokenizer plus Transformer

The original CTTS paper Zeng et al. 2023 motivates the CNN-plus-Transformer combination from two complementary inductive biases. A 1D CNN with local kernels captures *short-term* dependencies in the sequence well: each token in the output of the convolutional stack is a function of a bounded receptive field of input trades, and the hierarchical pooling structure compresses the sequence to a coarser temporal resolution while increasing channel width to preserve information. The CNN therefore acts as a learnable tokenizer that maps raw, high-frequency trade-level inputs into a small number of dense, information-rich tokens. A vanilla Transformer encoder operating directly on the 64 raw trades would have 64 tokens and would need to learn local trade-to-trade interactions through

attention, an inefficient allocation of attention capacity when those interactions are overwhelmingly local; by precomputing local features with a CNN, the Transformer can focus its attention budget on long-range, cross-token dependencies. A Transformer encoder over the tokenized sequence captures *long-term* dependencies symmetrically across the sequence: every token attends to every other token, so the model can detect arbitrary non-local patterns (e.g. a burst of activity 60 trades ago that correlates with the most recent ten trades) that a strictly causal recurrence like Mamba or the LSTM would have to propagate through its hidden state.

For our setting this inductive bias is well matched to the data. Kalshi trade sequences exhibit both short-range microstructure (consecutive trades on the same ticker form contiguous bursts with characteristic price-and-volume signatures) and long-range correlations (the burst that ended 60 trades ago is informative for the burst that begins now). The CNN captures the first scale, the Transformer captures the second, and the `[CLS]` token gives the model an explicit, learnable summary position to focus attention on, in contrast to the implicit "last hidden state" summary used by Mamba and the LSTM.

A property worth noting is that CTTS departs from the strictly causal discipline of Mamba and the LSTM: the Transformer encoder uses full bidirectional self-attention across the 17-token sequence (the `[CLS]` plus 16 CNN tokens), so the `[CLS]` representation at the output is a function of all 16 CNN tokens, not just those preceding it in time. Because the CNN's tokens are themselves causal features of the input trades (each token covers a contiguous block of trades earlier than the prediction point), and because the prediction target is a function of trades *after* the window, the model does not see any future information. The bidirectional attention simply gives the model the freedom to integrate evidence from earlier and later positions within the window when summarising it. This design trades the strict causality of Mamba and the LSTM for greater representational flexibility within the window.

F.3 Dataset Construction and Subsampling

The CTTS dataset reuses the per-trade feature matrix and the temporal 80/20 split from the rest of this paper, but it differs in three ways from the sequence-dataset construction used in Appendices C and D.

First, the window length is $L = 64$ rather than 32. This was a deliberate choice motivated by the architecture: with three pooling stages, an input of length 64 produces 16 output tokens, which is a reasonable sequence length for a Transformer encoder. A length-32 input would produce only 8 tokens, which we judged too few to merit the attention machinery; a longer input would produce more tokens but would also push training time per epoch above what was feasible in a single Colab session.

Second, the training, validation, and test sets are stratified random subsamples of $800{,}000$, $160{,}000$, and $160{,}000$ windows respectively, drawn independently for each horizon with a horizon-specific random seed. The full per-horizon window set ranges from 24M to 28M windows; processing them all would require several Colab sessions per horizon, exceeding our available compute budget. The subsampling is stratified to preserve class proportions, which range from $\sim 8\%$ jumps each direction at the 5-minute horizon to $\sim 11\%$ at 60m, mirroring the proportions of the full sets. Adjacent windows in the per-trade dataset share 63 of 64 input rows, so the information loss from subsampling is much smaller than the $30 \to 1$ reduction in row count suggests: most of the discarded windows are near-duplicates of retained windows.

Third, per-window normalisation is applied to `yes_price`: within each window the price column is rescaled to $[0, 1]$ via min-max, following the CTTS paper's preprocessing recipe. All other features are z-scored once using training-set statistics. The per-window normalisation removes the absolute price level from the model's input and forces it to learn from shape rather than level; this is a well-justified preprocessing choice for percent-return forecasting on unbounded prices but is a less obvious choice on Kalshi's bounded $[0, 100]$-cent grid, where the absolute price level carries meaningful information about boundary effects and the resolution-implied probability. We retain the per-window scaling for fidelity to the original CTTS recipe and treat the ablation against z-scoring as future work.

The same memory-management techniques described in Appendix E apply here. The per-trade feature matrix is held once in RAM as a contiguous `float32` array, and window retrieval is a fancy-index gather, so the dataset object is small enough to be replicated across DataLoader workers without

OOM. We use `NUM_WORKERS = 4` and `pin_memory = True` on T4; on single-GPU L4 or A100, additional workers do not noticeably improve throughput because the bottleneck is GPU compute rather than data loading.

F.4 Training Objective

Each horizon model is trained with class-weighted softmax cross-entropy on the three-class labels $y_t^{(h)} \in \{-1, 0, +1\}$ (shifted to $\{0, 1, 2\}$ for PyTorch). The loss is

$$\mathcal{L}^{(h)} \;=\; -\frac{1}{N}\sum_{i=1}^{N} w^{(h)}_{y_i^{(h)}} \log p^{(h)}_{y_i^{(h)}}(\mathbf{x}_i),$$

where the class weights $w_c^{(h)}$ are computed with a power-0.5 inverse-frequency rule:

$$w_c^{(h)} \;=\; \left(\frac{N^{(h)}_{\text{fit}}}{3N_c^{(h)}}\right)^{0.5}, \qquad \text{normalised so that} \quad \tfrac{1}{3}\sum_c w_c^{(h)} = 1.$$

This is a softening of the linear inverse-frequency rule $w_c^{(h)} = N^{(h)}_{\text{fit}}/(3N_c^{(h)})$ used by every other model in this paper. In an early CTTS run with linear weights, the 5-minute model achieved a balanced accuracy of 0.33, identical to random guessing across the three classes: the loss had been so heavily tilted toward the minority classes that the model collapsed onto a "always predict down" degenerate solution. The square-root variant preserves a meaningful penalty on majority-class errors while still incentivising the minority classes, and produced stable training across all four horizons. The class weights produced by this rule for the 60-minute horizon are $[1.24, 0.45, 1.31]$ on the $\{-1, 0, +1\}$ ordering, compared to $[2.66, 0.45, 3.30]$ under the linear rule.

Training proceeds with AdamW ($\beta = (0.9, 0.999)$, weight decay 10^{-4}), a cosine learning-rate schedule decaying from 3×10^{-4} to zero over 25 epochs, batch size 256, and gradient norm clipping at 1.0. Automatic mixed precision was disabled following recurrent `NaN` loss values at the 5-minute horizon in early runs; we did not isolate whether the cause was the batch-norm statistics, the softmax in attention, or the cross-entropy itself, but disabling `float16` resolved the issue across all four horizons at the cost of roughly 30% slower training. Validation macro-F1 is computed at the end of each epoch on a held-out $160{,}000$-window validation fold carved chronologically off the end of the training partition; training halts when no improvement is observed for 3 consecutive epochs, at which point the checkpoint corresponding to the best observed validation macro-F1 is restored for test-set evaluation. Total training time on a T4 GPU is approximately 45 minutes for all four horizons combined, the smallest training budget of any sequence model in this paper.

G FT-Transformer

G.1 Architecture

The FT-Transformer model, `FTTransformer`, is a tabular deep learning classifier introduced by Gorishniy et al. Gorishniy et al. 2021 and applied here as a per-trade flat-feature classifier. Unlike the sequence models of Appendices C, D, and F, the FT-Transformer consumes no temporal context directly: each trade is represented by the same 40-dimensional feature vector described in Section 4.4, and all multi-trade history is condensed a priori into the engineered backward-return, recent-jump-count, and rolling-window features. The model tokenizes each numerical feature into a learned d-dimensional embedding via a linear projection (the Feature Tokenizer), prepends a learnable `[CLS]` token, and processes the resulting 41-token sequence with a standard Transformer encoder Vaswani et al. 2017. The `[CLS]` representation at the output is passed to a three-way classification head. Four such models are trained independently, one per horizon, on the full 51M-row training set. Figure 13 gives a schematic of the full forward pass.

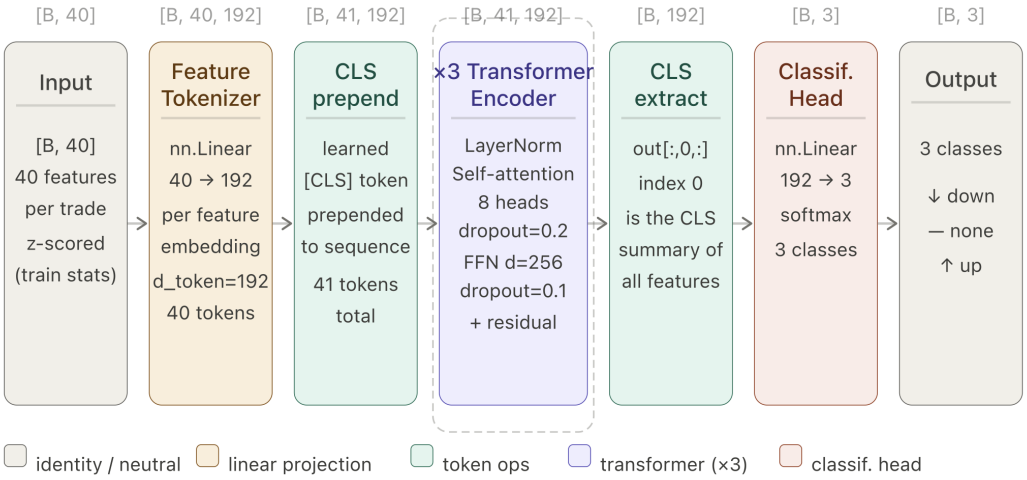

Figure 13: **FT-Transformer architecture for one forecast horizon.** Each trade's 40-dimensional feature vector is independently projected into a $d_{\text{token}} = 192$ embedding by the Feature Tokenizer. A learned `[CLS]` token is prepended, yielding a sequence of 41 tokens. Three Transformer encoder layers with 8-head self-attention, FFN width 256, and residual connections process the full token sequence. The `[CLS]` representation at position 0 is extracted and passed through a linear head producing logits over the three output classes. Four such models are trained independently, one per horizon.

The Feature Tokenizer maps each of the $F = 40$ numerical input features independently into a $d_{\text{token}} = 192$-dimensional vector via a learned linear projection with bias:

$$\mathbf{T}_f \;=\; x_f \cdot \mathbf{w}_f + \mathbf{b}_f, \qquad \mathbf{w}_f, \mathbf{b}_f \in \mathbb{R}^{d_{\text{token}}},$$

producing a token matrix $\mathbf{T} \in \mathbb{R}^{B\times 40\times 192}$. A learnable `[CLS]` token $\mathbf{c} \in \mathbb{R}^{d_{\text{token}}}$ is prepended to yield $\hat{\mathbf{T}} \in \mathbb{R}^{B\times 41\times 192}$. This sequence is processed by $N = 3$ standard Transformer encoder layers, each applying pre-LayerNorm multi-head self-attention ($H = 8$ heads, attention dropout 0.2) followed by a position-wise feed-forward network ($d_{\text{ff}} = 256$, FFN dropout 0.1) with a residual connection. After the encoder, the `[CLS]` token at position 0 is extracted:

$$\mathbf{h}_{\text{cls}} \;=\; \hat{\mathbf{T}}_{\text{out}}[:,\, 0,\, :] \;\in\; \mathbb{R}^{B\times 192},$$

and mapped to three logits by a linear classification head $\mathbf{W}_{\text{head}} \in \mathbb{R}^{192\times 3}$. A softmax converts these to class probabilities. The total parameter count is approximately 3.6M per horizon model, the smallest of the neural baselines in this comparison.

Table 11: FT-Transformer hyperparameters, identical across all four horizon models.

Hyperparameter	Value	Notes
N_NUM_FEATURES	40	Input feature dimension
D_TOKEN	192	Feature embedding dimension
N_BLOCKS	3	Transformer encoder layers
ATTENTION_N_HEADS	8	Self-attention heads
ATTENTION_DROPOUT	0.2	Attention weight dropout
FFN_D_HIDDEN	256	Feed-forward inner dimension
FFN_DROPOUT	0.1	Feed-forward dropout
RESIDUAL_DROPOUT	0.0	Residual stream dropout
D_OUT	3	Output classes (down / none / up)
BATCH_SIZE	8,192	Mini-batch size
N_EPOCHS	3	Training epochs
LR	1e-4	AdamW peak learning rate
WEIGHT_DECAY	1e-5	AdamW weight decay
SCHEDULER	OneCycleLR	One-cycle with 10% warmup
GRAD_CLIP	1.0	Max gradient norm
DROP_LAST	True	Avoids off-by-one in OneCycleLR

G.2 Why Feature Tokenization for Tabular Data

Classical deep learning architectures (MLPs, CNNs, RNNs) treat all input features symmetrically, either concatenating them into a flat vector before processing or stacking them along a channel dimension. This imposes no structure on the relationship between features: the network must discover, through its weights, that `yes_price` and `dist_from_50` encode related information, or that the eight `ticker_emb` dimensions belong to a coherent semantic group. Gradient-boosted trees sidestep this problem by learning axis-aligned splits on individual features, which naturally gives each feature its own decision boundary; the FT-Transformer achieves an analogous effect through the Feature Tokenizer.

By projecting each feature independently into a d-dimensional vector before attention, the Feature Tokenizer assigns each feature its own learnable representation that is not entangled with the representations of other features at the input layer. The Transformer encoder then learns *interactions* between these per-feature representations through self-attention: the attention weight α_{ij} measures how informative feature j's token is for updating feature i's token given the current input. This is precisely the kind of conditional feature interaction that tabular prediction tasks require Gorishniy et al. 2021: whether `recent_up_jumps_5m` is informative depends on the value of `yes_price` and the values of the `ticker_emb` dimensions, and the attention mechanism can learn these conditional dependencies directly.

Empirically, Gorishniy et al. show that the FT-Transformer matches or exceeds gradient-boosted trees on a suite of tabular benchmarks, and that the Feature Tokenizer is the key architectural component responsible for this performance: replacing it with a shared linear projection (as in the original Transformer) degrades performance to below MLP baselines. Our results are broadly consistent with this finding: the FT-Transformer achieves macro F1 within 1–2 points of LightGBM across all horizons despite consuming no temporal context, and it matches or outperforms the LSTM and Mamba sequence models that do have access to a 32-trade window.

G.3 Dataset Construction and Challenges

We reused the temporal 80/20 split established in preprocessing, with the first 80% of trades (by timestamp) forming the training set and the remaining 20% the test set. Unlike the sequence models, the FT-Transformer does not require windowing: every trade in the training set is a valid training example, giving the model access to all 51M training rows without the coverage loss that windowing imposes (the sequence models evaluate on ~ 537K non-overlapping windows, roughly 4% of the available test trades).

The primary engineering challenge at this scale was memory-efficient data loading. The full feature matrix at `float32` occupies approximately 8.2 GB for training and 2.0 GB for test. We addressed this by storing the normalised feature matrices and label arrays as memory-mapped `.npy` files on Google Drive, reading them via `np.memmap` with `mode='r'` rather than loading them into RAM. Each DataLoader worker reads a small contiguous slice from the memmap at each step, keeping peak RAM usage bounded regardless of dataset size. The DataLoader was configured with `drop_last=True` to ensure that the number of batches per epoch is exactly `len(loader)`, which is required for the `OneCycleLR` scheduler whose `total_steps` must match the actual training step count; failure to set `drop_last=True` caused an off-by-one `ValueError` in an earlier run that crashed at the final step of the last epoch.

Class imbalance was handled via per-horizon inverse-frequency weights computed on the training fold:

$$w_c^{(h)} \;=\; \frac{N_{\text{train}}^{(h)}}{3 \cdot N_c^{(h)}},$$

passed as the `weight` argument to `nn.CrossEntropyLoss`. The weights were estimated from a 5M-row random sample of the training labels rather than the full 51M rows for computational efficiency; at this sample size the estimates are stable to three decimal places.

G.4 Training Objective

Each horizon model is trained independently with class-weighted softmax cross-entropy on the three-class labels $y_t^{(h)} \in \{0, 1, 2\}$ (re-indexed from $\{-1, 0, +1\}$ for PyTorch):

$$\mathcal{L}^{(h)} \;=\; -\frac{1}{N}\sum_{i=1}^{N} w_{y_i^{(h)}}^{(h)} \,\log p_{y_i^{(h)}}^{(h)}(\mathbf{x}_i), \qquad p_c^{(h)}(\mathbf{x}) \;=\; \frac{\exp z_c^{(h)}(\mathbf{x})}{\sum_{c'} \exp z_{c'}^{(h)}(\mathbf{x})},$$

where $z_c^{(h)}(\mathbf{x}) = \mathbf{W}_{\text{head}}^{(h)}\,\mathbf{h}_{\text{cls}}^{(h)}(\mathbf{x})$ is the class-c logit produced by the classification head applied to the `[CLS]` token of the FT-Transformer. Unlike the Mamba, LSTM, and Moirai models, which train a single shared backbone with four horizon-specific heads, the FT-Transformer trains four entirely independent models, one per horizon. This is consistent with the LightGBM setup and reflects the observation that the four horizons have different optimal feature-attention patterns (as visible in the gradient-saliency plots of Section **??**), making parameter sharing across horizons potentially harmful.

Training proceeds with AdamW ($\beta = (0.9, 0.999)$, weight decay 10^{-5}), a `OneCycleLR` schedule with peak learning rate 10^{-4}, 10% linear warmup, and cosine annealing to zero over three epochs, batch size $8{,}192$, and gradient norm clipping at 1.0. Three epochs over 51M rows at batch size $8{,}192$ corresponds to approximately $18{,}690$ gradient steps per epoch, or $56{,}070$ total steps per model. Training each horizon model takes approximately one hour on a single NVIDIA A100 SXM4 80GB GPU. No early stopping was applied; the model from the final epoch was used for evaluation in all cases, as validation loss was still decreasing at epoch 3 and further training was precluded by compute budget.

G.5 Feature Importance

Unlike gradient-boosted trees, the FT-Transformer does not expose a natural per-feature importance score. We therefore computed gradient saliency on a random subsample of $50{,}000$ test trades: for each sample, we computed the absolute gradient of the class probabilities with respect to the input feature vector, summed across all three classes, then averaged across samples. This yields a mean $|\nabla_{\mathbf{x}} p|$ per feature, interpreted as the average sensitivity of the model's output to a marginal perturbation of that feature.

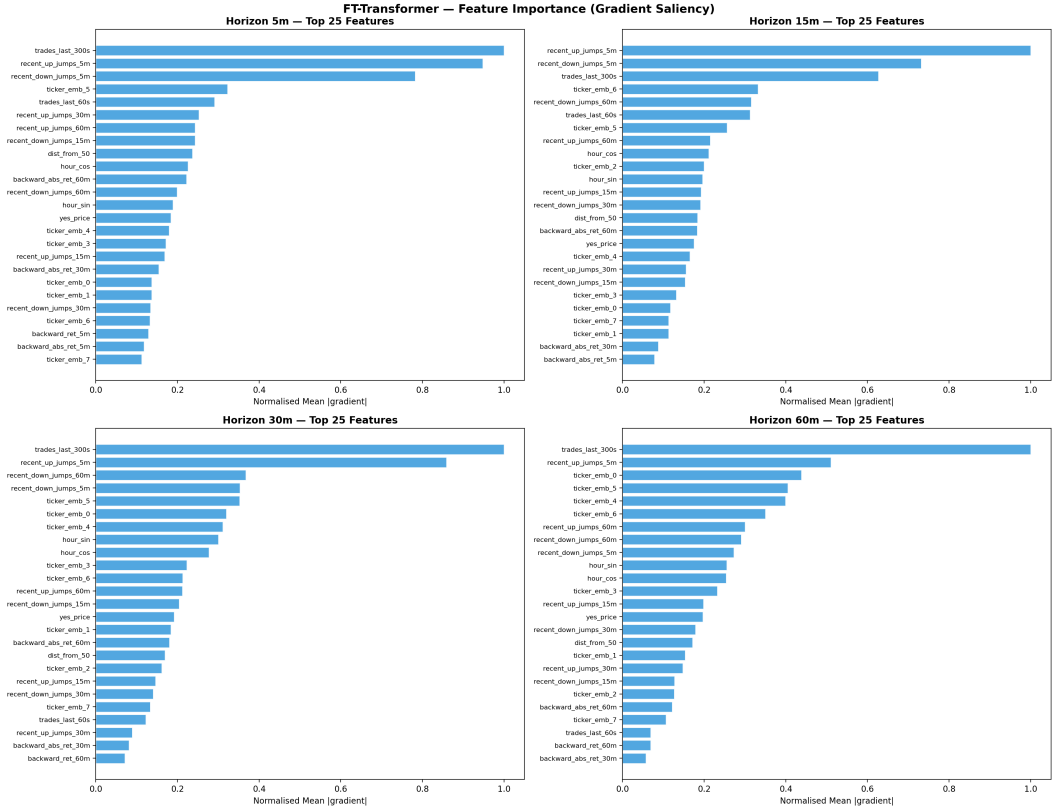

Figure 14: Top 25 features by gradient saliency for the FT-Transformer at each forecast horizon, computed on a 50,000-trade random subsample of the test set. Importance is measured as mean $|\nabla_{\mathbf{x}} p|$ summed across all three output classes and normalised to $[0, 1]$ within each horizon for readability. The dominance of `trades_last_300s` and the 5-minute jump-count features, and the growing importance of ticker embeddings at longer horizons, are both visible.

G.6 Confusion Structure

Figure 15: Row-normalised confusion matrices for the four per-horizon FT-Transformer models on the full 12.76M-row test set. Each cell shows the fraction of the true class (large) and the underlying count (small). Diagonal elements correspond to correct predictions. Note the near-perfect symmetry of cross-directional errors at 5m (38.5% of true down-jumps predicted as up and vice versa), attenuating at longer horizons as price-level and market-identity features become more directionally discriminative.

H Mixture of Experts

H.1 Architecture

The MoE gating network, `MoEGate`, builds on the mixture-of-experts framework originally proposed by Jacobs et al. 1991, in which a learned gating function combines the outputs of multiple specialized subnetworks. We adopt the soft gating mechanism of Shazeer et al. 2017, where all experts contribute to the final prediction with weights determined by a softmax over gate logits, rather than a hard top-k routing decision. Crucially, the architecture constrains the final prediction to be a convex combination of the six expert probability distributions: the gate cannot express directional opinions that no expert holds, only adjust how much weight to place on each expert at each input.

The forward pass has three stages. First, the gate feature vector $\mathbf{x} \in \mathbb{R}^F$ is passed through a three-layer MLP backbone with hidden dimensions $(192, 96, 48)$. Each layer consists of a linear projection, LayerNorm, GELU activation, and dropout ($p = 0.3$). Second, the backbone output is projected by a linear head to produce $M = 6$ raw gate logits, one per expert. Third, unavailable experts (those with no prediction for the current row) are masked out by setting their logits to -10^9 before a softmax normalises the weights:

$$\mathbf{g} = \mathrm{softmax}\big(\mathrm{MaskFill}(\mathbf{W}_g\mathbf{h},\ \boldsymbol{\delta} = 0,\ -10^9)\big) \ \in\ \Delta^{M-1},$$

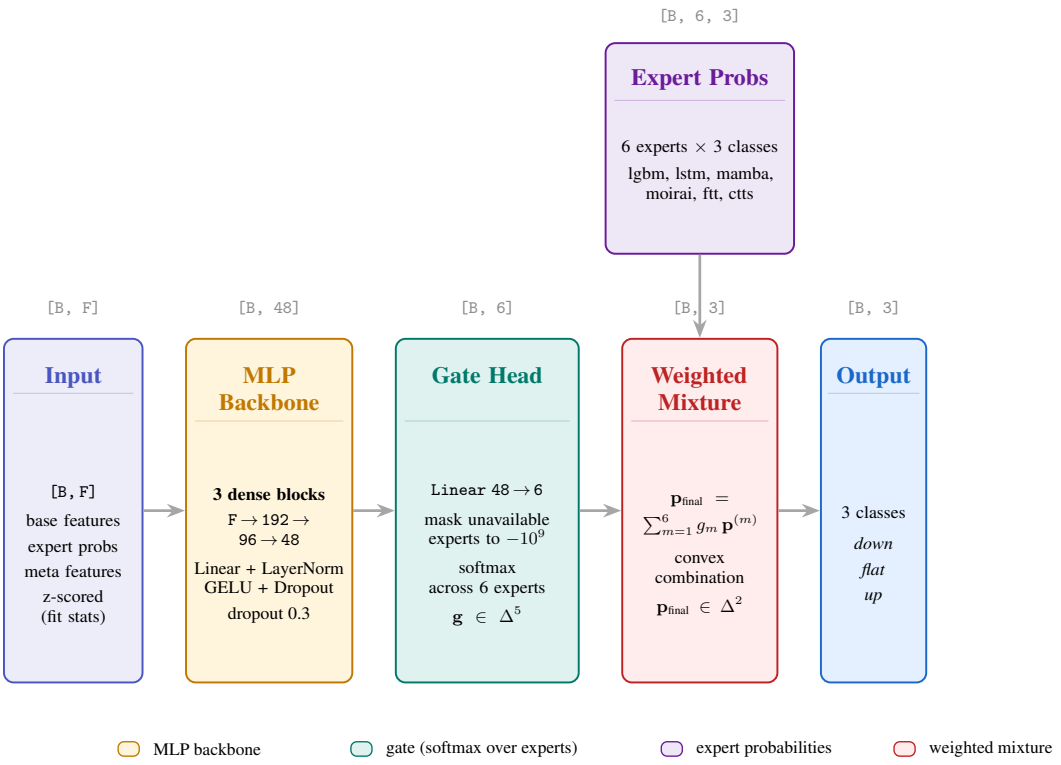

Figure 16: **MoEGate architecture for one forecast horizon.** A per-trade feature vector $\mathbf{x} \in \mathbb{R}^F$ combining base trade features, the six experts' probability distributions, and horizon-specific meta-features is passed through a three-layer MLP backbone with LayerNorm and GELU activations. A linear gate head produces six raw logits, one per expert; unavailable experts (mamba, moirai, and ctts have non-trivial coverage gaps) are masked to -10^9 before softmax, ensuring they receive zero weight. In parallel, the six expert probability vectors flow directly into the weighted-mixture stage, where the gate weights produce the final convex combination. Four such models are trained independently, one per horizon.

where $\boldsymbol{\delta} \in \{0,1\}^M$ is the expert availability mask and $\mathbf{h}$ is the backbone output. The final class probabilities are computed as a convex combination of the experts' predictions:

$$\mathbf{p}_{\text{final}} = \sum_{m=1}^{M} g_m \cdot \mathbf{p}^{(m)}, \qquad \mathbf{p}^{(m)} \in \Delta^2,$$

where $\mathbf{p}^{(m)}$ is the three-class probability vector from expert m over $\{\text{down}, \text{no jump}, \text{up}\}$. For missing expert predictions, $\mathbf{p}^{(m)}$ is set to the uniform distribution $[1/3, 1/3, 1/3]$ and the corresponding availability mask entry is zero, ensuring the gate ignores that expert entirely.

H.2 Why a Soft Gating Mechanism

The central motivation for a learned soft gate rather than a fixed ensemble average is that different experts have heterogeneous strengths that depend on the current market context Rokach 2010. From Table 4, LightGBM is consistently strong on macro F1 but conservative on directional jumps; Moirai achieves higher balanced accuracy at longer horizons but over-predicts minority classes; CTTS leads on short-horizon weighted F1. A fixed average blends these profiles indiscriminately. The gate can learn, for instance, to up-weight Moirai when the rolling-accuracy meta-features indicate it has been recently reliable on the current ticker, or to down-weight an expert whose recent accuracy has dropped below the random-guess baseline of $1/3$.

A soft gate is also more appropriate than a hard top-k routing strategy in our setting. Hard routing introduces discontinuities in the loss surface and amplifies the impact of variable expert availability

Table 12: MoE gating network hyperparameters, identical across all four horizon models.

Hyperparameter	Value	Notes
HIDDEN_DIMS	(192, 96, 48)	MLP backbone layer widths
DROPOUT	0.3	Applied after each GELU
N_EXPERTS	6	lgbm, lstm, mamba, moirai, ftt, ctts
N_CLASSES	3	down / no jump / up
BATCH_SIZE	4096	Mini-batch size
LR	1e-3	AdamW learning rate
WEIGHT_DECAY	0.01	AdamW weight decay
MAX_EPOCHS	30	Upper bound on training
PATIENCE	3	Early stopping on val NLL
WARMUP_STEPS	500	Linear warmup then cosine decay
ENTROPY_LAMBDA	0.01	Entropy regularisation coefficient
GRAD_CLIP	1.0	Max gradient norm

near ticker boundaries (where mamba, moirai, and ctts frequently lack predictions). The soft combination remains differentiable and degrades gracefully: when an expert is unavailable the gate redistributes its weight to the remaining experts via the softmax mask, with no architectural change required at inference time.

The convex-combination constraint is the key architectural distinction between this approach and a more permissive meta-classifier such as a stacked LightGBM, which we considered as an alternative. The restriction is real: the MoE cannot express a class probability that no expert assigned non-trivial weight to, so it cannot recover from unanimous expert errors. In exchange, the gate weights themselves carry interpretable meaning–"the gate trusts model m with weight g_m at this input"–which a black-box meta-classifier does not provide. For the purposes of this work, the interpretability of which experts dominate at which horizons and market regimes is itself a primary contribution, justifying the constraint.

H.3 Training Objective

The gate is trained with a composite loss combining class-weighted negative log-likelihood and an entropy regularisation term following Pereyra et al. 2017:

$$\mathcal{L} = \underbrace{-\sum_{c} w_c \log p_{\text{final},c}}_{\text{weighted NLL}} - \lambda \underbrace{\left(-\sum_{m=1}^{M} g_m \log g_m \right)}_{\text{gate entropy}},$$

where $w_c = N_{\text{fit}}/(3N_c)$ are the inverse-frequency class weights computed on the fit fold and $\lambda = 0.01$ is the entropy coefficient. The class weighting is identical in form to that used by the standalone models, preserving consistency across the pipeline.

The entropy term encourages the gate to distribute weight across multiple experts rather than collapsing onto a single one early in training. Without it, the gate tends to collapse onto whichever expert has the lowest training loss in the first few epochs, producing a degenerate solution that ignores the remaining five experts. This failure mode is well-documented in the ensemble learning literature Jacobs et al. 1991; Rokach 2010. With $\lambda = 0.01$, the entropy bonus is small relative to the NLL ($\sim 1\%$ of typical loss magnitudes in our experiments) but large enough to keep all six experts in play during early training, after which the NLL gradient takes over and shapes the differentiated routing visible in the gate-weight visualisations.

Validation NLL is computed without the entropy term and monitored after each epoch; training halts when no improvement is observed for three consecutive epochs, at which point the checkpoint corresponding to the lowest observed validation NLL is restored for downstream evaluation.

H.4 Gate Input Features

The gate receives three categories of input for a given horizon h. First, the 41 base trade-level features from Section 4.4, providing the same market context available to the standalone models. Second, the six expert probability vectors $\{\mathbf{p}^{(m)}\}_{m=1}^{6}$, each a three-dimensional distribution over jump classes, giving the gate direct access to each expert's current prediction. Third, the 19 MoE meta-features per horizon described in Appendix A.2, comprising rolling per-expert accuracies at two window lengths ($w \in \{50, 1000\}$, a total of 12 features), six expert availability indicators, and a modal-class agreement score across the six experts. Features from other horizons $h' \neq h$ are explicitly excluded to prevent cross-horizon leakage, yielding 78 input features per horizon model.

Rolling-accuracy columns that are `NaN` due to insufficient ticker history are filled with $1/3$ rather than the training-fold mean, to avoid inflating perceived reliability for new markets where no prior accuracy can be observed. All remaining features are z-scored using fit-fold statistics only; any residual `NaN` or `inf` values are set to zero after normalisation, which after z-scoring corresponds to the train-fold mean–a benign imputation that does not introduce predictive bias.

H.5 Dataset Construction

The MoE dataset is constructed from a pre-merged parquet file (`moe_data.parquet`) that joins the base trade features with the prediction outputs of all six expert models and the pre-computed meta-features from Appendix A.2. The merge pipeline itself was non-trivial: of the six expert prediction dataframes, two (lstm/lgbm and fft) were row-aligned positionally to the features and could be concatenated directly after explicit alignment verification on all rows, while three (mamba, ctts, moirai) had dropped rows from upstream pipeline differences and were combined via mean aggregation within each `(ticker, created_time)` group followed by left merge.

Two construction decisions deserve emphasis. First, rows where any of the six experts lacked a prediction were dropped from the MoE dataset entirely, restricting evaluation to the intersection of all six experts' coverage–approximately 60–65% of the original test trades. This is a tighter sample than the standalone models were evaluated on, and the resulting MoE metrics are not directly comparable to base-model metrics on the full test set; we report both for clarity.

Second, the train/test split is anchored to the same chronological boundary used by the base models (the 80th-percentile timestamp of the original dataset), not to the 80th-percentile timestamp of the filtered MoE dataset. This preserves the temporal boundary across all model comparisons and prevents the dropped rows from inadvertently shifting the effective test set. Within the training partition, the last 10% by timestamp is carved off as a validation fold for early stopping. One model is trained per horizon by setting `HORIZON` $\in \{5, 15, 30, 60\}$.

H.6 Evaluation

After loading the best checkpoint, the gate is evaluated on the filtered test set using the same metrics as the standalone models: balanced accuracy, macro F1, weighted F1, and per-class precision and recall. The mean gate weight per expert across the test set provides an interpretability diagnostic, indicating which experts the gate relies on most heavily in the out-of-sample period.

We additionally visualise the gate weights as a function of time over the test period, resampled to hourly bins to smooth across the ticker-interleaved trade ordering. Stacked-area plots of these hourly weights reveal whether routing is stable across the test window or shifts meaningfully with market regimes–for example, whether sports-event days concentrate weight on one expert relative to politics-heavy days. Permutation importance on the gate inputs identifies which features most influence routing decisions: a healthy gate should rely heavily on the rolling-accuracy meta-features (an indication that it has learned context-dependent expert trustworthiness) rather than only on the expert probabilities themselves.

H.7 Feature Importance

To understand which inputs the MoE gate relies on most when routing trades to experts, we compute permutation importance on a 200,000-row sample of the held-out test set: for each input feature in turn, we shuffle its values across the sample, recompute balanced accuracy, and report the resulting

drop. The procedure is repeated twice per feature and the mean drop reported. Figure 17 shows the top 25 features by permutation drop for each horizon.

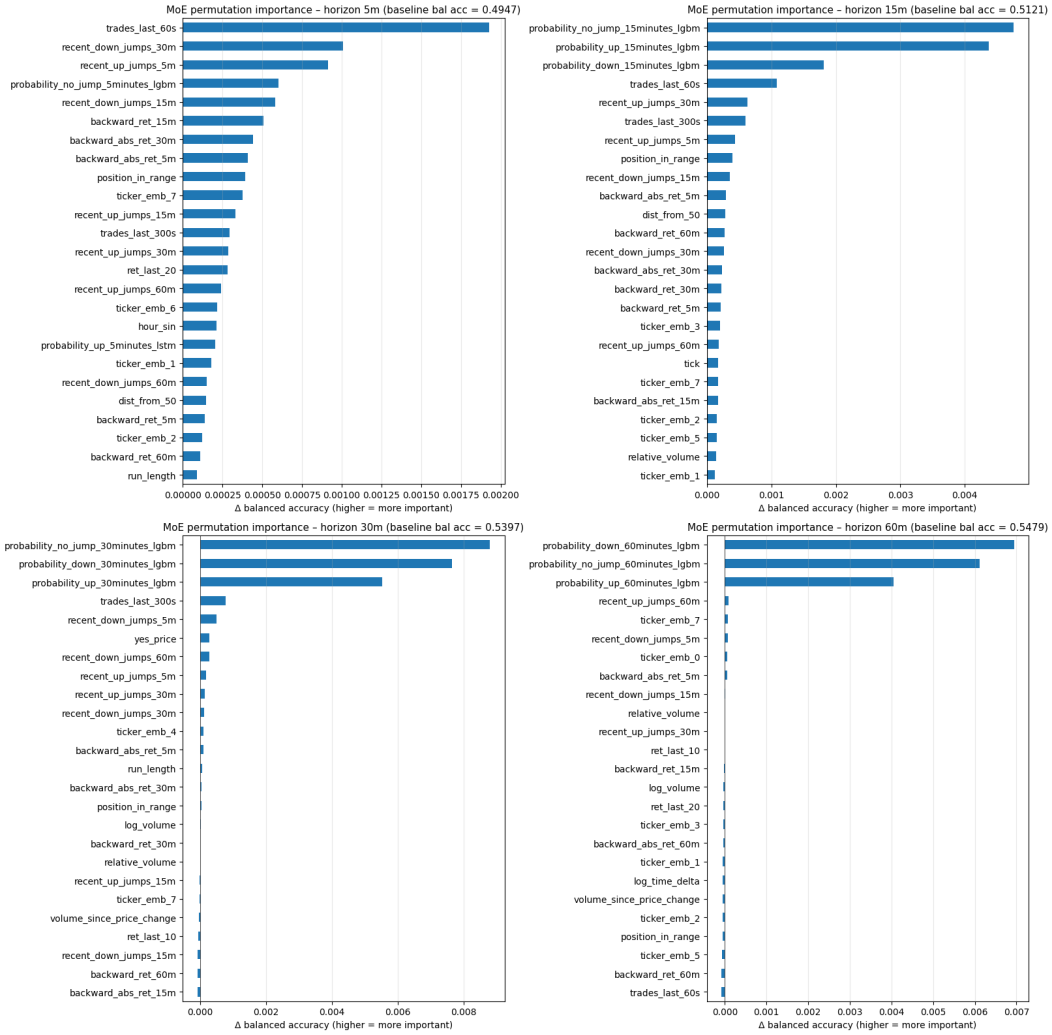

Figure 17: **MoE gate permutation importance per horizon.** For each feature, the bar shows the drop in balanced accuracy when that feature is shuffled across the test sample. At the 5-minute horizon the gate relies on a broad mix of trade-microstructure features (`trades_last_60s`, `recent_*_jumps_*m`, `backward_ret_15m`) with only modest dependence on individual expert probabilities. From 15 minutes onward, LightGBM's probability columns (`probability_{down,no_jump,up}_{15,30,60}minutes_lgbm`) sweep the top three slots, indicating that the gate's routing decisions at longer horizons are dominated by reading off LightGBM's predicted distribution, consistent with the gate-weight distributions of Figure 22, which show LightGBM as the dominant expert at every horizon.

H.8 Expert Routing Over Time

A primary interpretability question for an MoE gate is whether routing is stable across time or shifts with market conditions. To investigate this we plot, for each horizon, the gate weights assigned to each expert as a function of wall-clock time over the test window. Because trades are interleaved across hundreds of tickers, raw per-trade weights are highly noisy; we therefore resample to hourly bins and apply a 6-hour rolling mean for visualisation. The resulting stacked-area plots show the proportional weight each expert receives at each point in time.

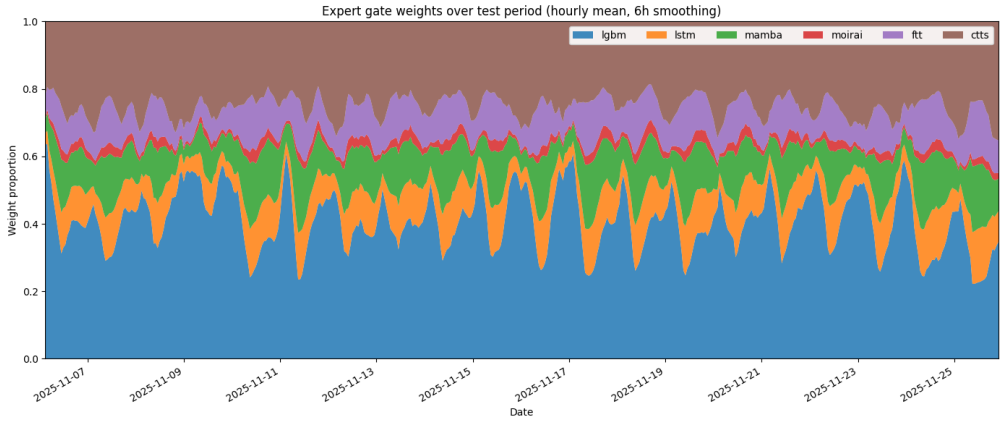

Figure 18: **Gate weights over the test period at the 5-minute horizon** (hourly mean, 6-hour smoothing). LightGBM holds the largest share throughout but with strong sub-daily oscillations between roughly 0.20 and 0.55, reflecting the gate redistributing weight to CTTS and FFT during periods of high trading activity. The diurnal pattern is visible in the regular dips and recoveries of LightGBM's weight.

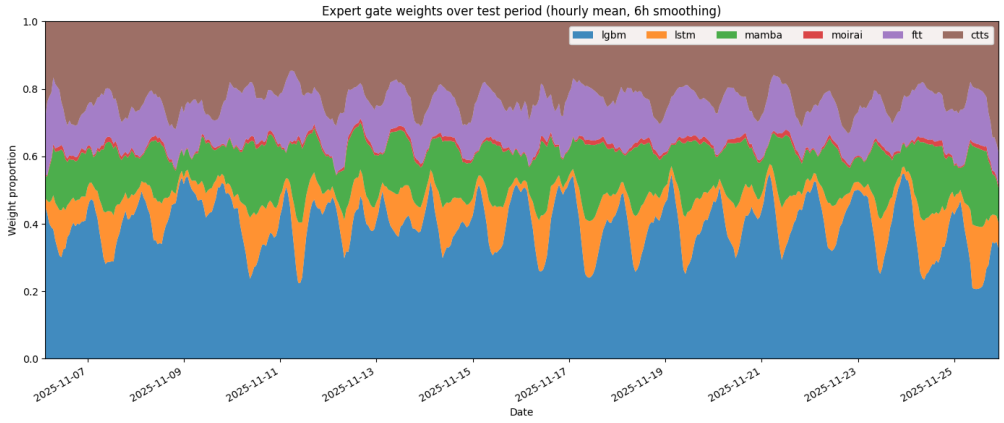

Figure 19: **Gate weights over the test period at the 15-minute horizon.** The diurnal pattern strengthens relative to the 5-minute horizon and CTTS receives noticeably more weight (the upper brown band expanding compared to 5m). Mamba and Moirai remain marginal contributors throughout, consistent with their lower standalone-model performance at these horizons.

H.9 Confusion Matrices

Figure 22 shows the row-normalised confusion matrices for the MoE gate at each horizon, evaluated on the filtered test set where all six experts have predictions.

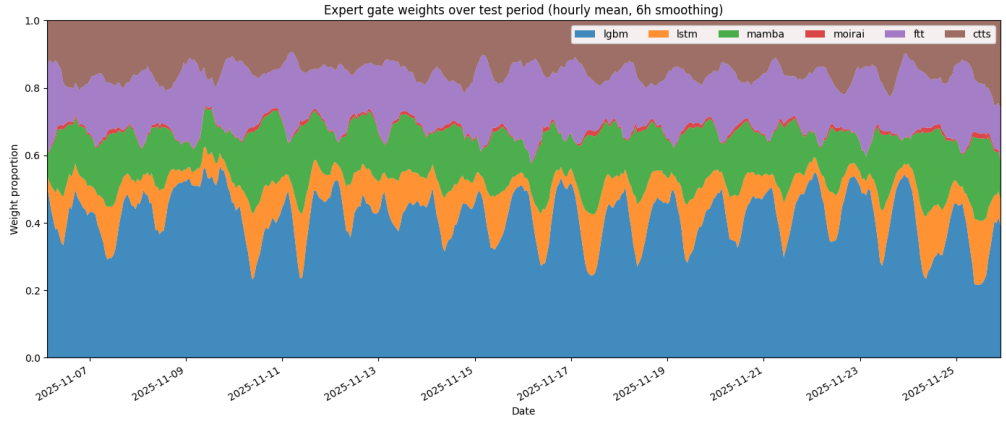

Figure 20: **Gate weights over the test period at the 30-minute horizon.** FFT (purple) takes on a substantially larger and more variable role here than at shorter horizons, with its share oscillating between roughly 0.05 and 0.20. Mamba's contribution also increases, particularly during periods of suppressed LightGBM weight. The diurnal cycle persists but is dampened compared to 15m, suggesting that longer-horizon predictions are less sensitive to intraday trade-frequency variation.

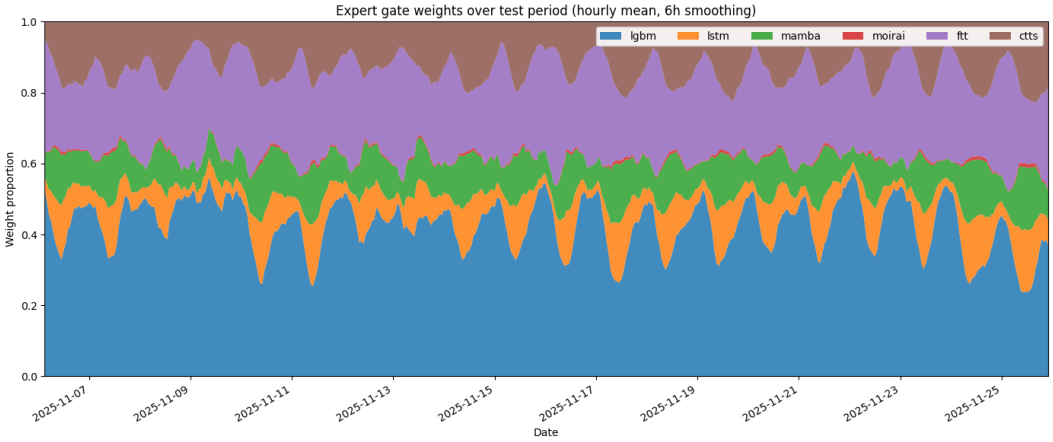

Figure 21: Expert gate weights over the test period (7–25 Nov 2025), hourly mean with 6-hour smoothing, at the 60-minute horizon. LightGBM dominates throughout; Moirai receives near-zero weight consistently.

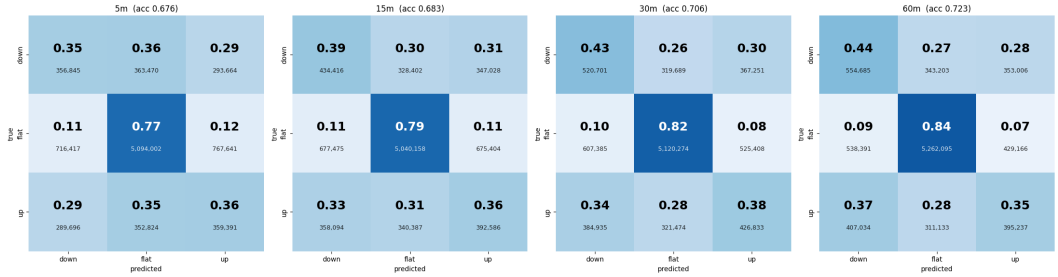

Figure 22: **Row-normalised confusion matrices for the MoE gate, per horizon.** Each cell shows the fraction of the true class (large) and the underlying count (small). The *flat* (no-jump) row is correctly classified at 0.77–0.84 across horizons, growing monotonically with horizon and substantially exceeding both the LSTM (0.61–0.69) and the LightGBM baseline (0.77–0.85). On the directional rows, the MoE recovers 35–44% of true down-jumps and 35–38% of true up-jumps, with directional-sign errors (down predicted as up, or vice versa) remaining a substantial fraction of the off-diagonal mass at every horizon, a pattern shared with both the LSTM and LightGBM baselines, indicating that combining experts does not resolve the directional confusion intrinsic to the underlying signal.